



Universidad de Jaén

Escuela Politécnica Superior de Jaén

Plataforma para la determinación de la explicabilidad de modelos de aprendizaje automático

Autor: Carlos Requena Doña

Grado: Ingeniería en Informática

Directores: Antonio Jesús Rivera Rivas y María Dolores Pérez Godoy
Departamento del director: Informática

Fecha: 31/7/2026

Licencia CC



CREA



UNIVERSIDAD DE JAÉN

D./D^a Antonio Jesús Rivera Rivas y D./D^a María Dolores Pérez Godoy, tutor(es) del Trabajo Fin de Grado titulado: **Plataforma para la determinación de la explicabilidad de modelos de aprendizaje automático**, que presenta Carlos Requena Doña, autoriza(n) su presentación para defensa y evaluación en la Escuela Politécnica Superior de Jaén.

Jaén, 31/7/2026

Agradecimientos

En primer lugar, quiero dar las gracias a mis tutores, Antonio Jesús Rivera Rivas y María Dolores Pérez Godoy, por su orientación, su disponibilidad y su paciencia a lo largo de todo el desarrollo de este trabajo, así como al grupo de investigación SIMI-DAT de la Universidad de Jaén por darme la oportunidad de contribuir a la plataforma *Nets4Learning*.

Gracias también a mi familia, a mi madre y a mi padre por el apoyo constante e incondicional durante estos años de carrera. En especial, gracias a Martín por estar ahí de forma incondicional en todo momento y ser mi mejor amigo, el mejor que uno puede pedir.

A Fran y Diego por estar ahí para hablar y echarnos unas risas tanto en Discord como cuando estamos juntos en persona.

Y a Mar, por ser mi refugio seguro constante. Por las risas compartidas cuando más lo necesitaba y por darme, sin saberlo, en uno de mis peores momentos la fuerza y la motivación para cruzar la línea de meta.

Resumen

Aunque los modelos de aprendizaje automático destacan a la hora de realizar predicciones, su naturaleza de “caja negra” limita de forma notable su utilidad en entornos educativos, donde entender el proceso es tan importante como el propio resultado. Este Trabajo Fin de Grado aborda esa opacidad diseñando e implementando una capa de Inteligencia Artificial Explicable (XAI) integrada de forma nativa en *Nets4Learning*, una plataforma educativa que se ejecuta en el navegador y está construida con React y TensorFlow.js.

En lugar de desarrollar nuevos algoritmos de predicción, el proyecto se centra por completo en hacer que los modelos que la plataforma ya ofrece resulten interpretables para el usuario final. La aportación principal es un sistema de explicabilidad que se ejecuta íntegramente en el lado del cliente y que combina dos enfoques complementarios, un método post-hoc agnóstico al modelo que calcula la contribución de cada característica a partir de la perturbación de la entrada mediante WebSHAP y un método específico (LRP) que propaga la relevancia capa a capa por el interior de la red. Para las tareas basadas en imágenes se apoya en algoritmos de segmentación en superpíxeles (SLIC) y de segmentación facial, con el fin de agrupar la entrada en regiones con significado, generando mapas de calor y gráficos de importancia que explican el comportamiento de la red.

Esta arquitectura de XAI se ha integrado con éxito en la detección de objetos, la clasificación de imágenes (incluidos los conjuntos MNIST y KMNIST) y los modelos tabulares. La implementación final respeta la restricción de la plataforma de funcionar sin servidor y se adapta a su interfaz multilingüe (español, inglés y japonés), dando como resultado una herramienta práctica que acorta la distancia entre las predicciones en bruto de los modelos y la comprensión humana en la enseñanza de la IA.

Palabras clave: Inteligencia Artificial Explicable, XAI, SHAP, LRP, aprendizaje profundo, TensorFlow.js, explicabilidad en el navegador, superpíxeles.

Abstract

While machine learning models excel at making predictions, their “black-box” nature significantly limits their usefulness in educational environments, where understanding the process is as critical as the result. This thesis tackles this opacity by designing and implementing an Explainable AI (XAI) layer natively into *Nets4Learning*, a browser-based educational platform built with React and TensorFlow.js.

Rather than developing new predictive algorithms, this project focuses entirely on making the platform’s existing models interpretable for the end user. The core contribution is an explainability system running purely client-side that combines two complementary approaches: a post-hoc, model-agnostic method that, by integrating WebSHAP, computes feature attributions via input perturbation without any server dependencies, and a model-specific method (LRP) that propagates relevance layer by layer through the network. For image-based tasks, it utilizes SLIC superpixel and facial segmentation algorithms to group inputs into meaningful semantic regions, generating visual heatmaps and importance charts that explain the network’s behavior.

This XAI architecture was successfully integrated across object detection, image classification (including MNIST/KMNIST datasets), and tabular models. The final implementation respects the platform’s zero-server constraints and seamlessly adapts to its multilingual interface (Spanish, English and Japanese), resulting in a practical tool that bridges the gap between raw model predictions and human comprehension in AI education.

Keywords: Explainable Artificial Intelligence, XAI, SHAP, LRP, deep learning, TensorFlow.js, in-browser explainability, superpixels.

Tabla de contenidos

1. Introducción	1
1.1. Contexto y Motivación	1
1.2. Objetivos	2
1.3. Estructura de la memoria	3
2. Especificación del trabajo	5
2.1. Descripción de la situación de partida	5
2.1.1. Descripción del entorno actual: Nets4Learning	5
2.1.2. Resumen de las deficiencias y carencias identificadas	6
2.2. Antecedentes	7
2.2.1. La Inteligencia Artificial Explicable (XAI)	7
2.2.2. Técnicas de explicabilidad	7
2.3. Restricciones	8
2.3.1. Restricciones temporales	9
2.3.2. Restricción de ejecución en el cliente	9
2.3.3. Restricciones de integración	9
2.4. Metodología de desarrollo software	10
2.5. Estimación del tamaño y el esfuerzo	12

2.5.1. Estimación del tamaño: líneas de código (LOC)	12
2.5.2. Estimación del esfuerzo: modelo COCOMO	13
2.6. Tecnologías utilizadas	14
2.6.1. Ejecución de modelos en el navegador	15
2.7. Planificación	15
2.8. Presupuesto	17
2.8.1. Costes de personal	17
2.8.2. Costes de hardware y software	17
2.8.3. Coste total	18
3. Análisis de requisitos	19
3.1. Requisitos funcionales	19
3.2. Requisitos no funcionales	19
3.3. Casos de uso	20
4. Diseño del sistema	25
4.1. Arquitectura general	25
4.2. Integración con Nets4Learning	26
4.3. Diseño del módulo de explicabilidad	27
4.3.1. Explicabilidad agnóstica mediante SHAP	27
4.3.2. Explicabilidad específica mediante LRP	28
4.4. Diseño de la visualización	28
4.5. Patrones de diseño aplicados	29
5. Desarrollo	31
5.1. Fundamentos comunes	31

5.1.1. Bibliotecas empleadas: WebSHAP y SLIC	31
5.1.2. Mecanismo general de SHAP	33
5.2. Incremento 1: explicabilidad en clasificación tabular	34
5.2.1. Objetivo del incremento	34
5.2.2. Diseño detallado	34
5.2.3. Implementación	35
5.2.4. Plan de pruebas	36
5.3. Incremento 2: explicabilidad en regresión	37
5.3.1. Objetivo del incremento	37
5.3.2. Diseño e implementación	37
5.3.3. Plan de pruebas	37
5.4. Incremento 3: explicabilidad en clasificación de imágenes	38
5.4.1. Objetivo del incremento	38
5.4.2. Diseño detallado	38
5.4.3. Implementación	40
5.4.4. Plan de pruebas	41
5.5. Incremento 4: explicabilidad en detección de objetos	42
5.5.1. Objetivo del incremento	42
5.5.2. Diseño detallado	42
5.5.3. Implementación	43
5.5.4. Plan de pruebas	45
5.6. Incremento 5: explicabilidad específica mediante LRP	45
5.6.1. Objetivo del incremento	45
5.6.2. Diseño detallado: fundamentos matemáticos	46

5.6.3. Implementación de las reglas de propagación	47
5.6.4. Captura de las activaciones y flujo de ejecución	50
5.6.5. Plan de pruebas	51
6. Conclusiones	53
6.1. Conclusiones	53
6.2. Líneas de trabajo futuro	54
Bibliografía	II

Lista de figuras

2.1. Modelo incremental aplicado al proyecto. Cada incremento añade la explicabilidad de una tarea y, dentro de él, se recorren las fases de análisis, diseño, implementación y pruebas.	11
2.2. Diagrama de Gantt con la planificación temporal del proyecto a lo largo de los dos cuatrimestres.	17
3.1. Diagrama de casos de uso del módulo de explicabilidad.	21
4.1. Arquitectura general del sistema con la capa de explicabilidad integrada.	26
4.2. Diagrama de secuencia de la generación de una explicación mediante SHAP.	28
5.1. Diagrama de clases del algoritmo SLIC.	32
5.2. Diagrama de clases del incremento de clasificación tabular.	34
5.3. <i>Storyboard</i> de la explicación tabular: predicción, explicación local y explicación global.	35
5.4. Diagrama de secuencia de la explicación de una clasificación de imágenes.	39
5.5. <i>Storyboard</i> de la explicación de imágenes: parámetros, cálculo con la galería de perturbaciones y mapas de calor por clase.	39
5.6. Flujo de una explicación en detección de objetos, con la elección de la segmentación.	43

5.7. <i>Storyboard</i> de la explicación en detección de objetos: detección original con los controles de la máscara y mapa de calor resultante.	45
5.8. Ilustración del funcionamiento de LRP.	46
5.9. Boceto del selector de método en la tarjeta de explicabilidad: con LRP disponible (a) y deshabilitado por no ser el modelo introspectable (b). .	47
5.10. Flujo de ejecución de una explicación mediante LRP.	52

Lista de tablas

2.1. Tamaño aproximado del módulo de explicabilidad, en líneas de código.	12
2.2. Estimación del esfuerzo mediante COCOMO básico (modo orgánico).	13
2.3. Planificación temporal del proyecto por fases e incrementos.	16
2.4. Estimación de costes de personal.	17
2.5. Estimación de costes de hardware y software.	18
2.6. Coste total estimado del proyecto.	18
3.1. Requisitos funcionales del sistema.	20
3.2. Requisitos no funcionales del sistema.	21
3.3. Caso de uso CU-1: explicación sobre datos tabulares.	22
3.4. Caso de uso CU-2: explicación de imagen mediante SHAP.	22
3.5. Caso de uso CU-3: explicación de imagen mediante LRP.	23
3.6. Caso de uso CU-4: configuración de los parámetros de la explicación.	23
4.1. Patrones de diseño aplicados en la solución.	29
5.1. Pruebas unitarias sobre las utilidades de muestreo de SHAP.	36

Capítulo 1

Introducción

La Inteligencia Artificial (IA) y, más concretamente, el Aprendizaje Profundo (*Deep Learning*), ha experimentado un crecimiento exponencial en la última década, integrándose en sectores críticos como la medicina, la automoción y la educación. Sin embargo, el aumento en la complejidad de estos modelos ha traído consigo un problema fundamental, la falta de transparencia. Las redes neuronales profundas actúan a menudo como “cajas negras”, ofreciendo predicciones precisas sin una justificación comprensible para el usuario humano.

En el ámbito educativo, esta opacidad es una barrera significativa. Para que un estudiante comprenda realmente cómo funciona una red neuronal, no basta con ver si acierta o falla; necesita entender qué características de los datos motivaron esa decisión.

1.1. Contexto y Motivación

Este Trabajo Fin de Grado (TFG) se desarrolla en el contexto del grupo de investigación SIMIDAT de la Universidad de Jaén. El punto de partida es la plataforma *Nets4Learning* ^{**1**}, una herramienta web diseñada para la enseñanza y experimentación con modelos de Deep Learning sin necesidad de hardware especializado ni conocimientos avanzados de programación.

La arquitectura de *Nets4Learning* es innovadora al ejecutar tanto el entrenamiento como la inferencia de los modelos en el lado del cliente (navegador) mediante Tensor-

¹Para facilitar la evaluación de este proyecto, se ha habilitado un prototipo funcional de demostración accesible en: <https://nets4learnings.netlify.app/>

Flow.js, lo que democratiza el acceso a la IA. Sin embargo, hasta ahora la plataforma se limitaba a mostrar la predicción del modelo, sin ningún mecanismo que explicase las decisiones tomadas.

La motivación principal de este proyecto es dotar a *Nets4Learning* de un módulo de Inteligencia Artificial Explicable (XAI). La condición principal es que dicho módulo debía respetar la filosofía de la plataforma, es decir, funcionar por completo en el navegador sin depender de ningún servidor que realizase los cálculos (backend). Esta restricción descarta las bibliotecas de explicabilidad más habituales, pensadas para ejecutarse en Python sobre un *backend*, y obliga a trabajar con las posibilidades que ofrece el cliente. Para abordarlo se han combinado dos enfoques complementarios, por un lado métodos agnósticos al modelo basados en la perturbación de la entrada, como SHAP, y por otro métodos específicos que analizan el interior de la red propagando la relevancia capa a capa, como LRP.

1.2. Objetivos

El objetivo general de este proyecto es diseñar, implementar e integrar un módulo de software que proporcione explicaciones visuales e interpretables de las predicciones realizadas por los modelos de la plataforma *Nets4Learning*, ejecutándose íntegramente en el navegador.

Para alcanzar esta meta, se han definido los siguientes objetivos específicos:

1. **Análisis de la arquitectura existente:** Estudiar el funcionamiento actual del *frontend* (React) y el motor de inferencia en cliente (TensorFlow.js) de *Nets4Learning*, así como el modo en que la plataforma gestiona sus modelos.
2. **Estudio y selección de técnicas XAI viables en cliente:** Analizar las distintas familias de técnicas de explicabilidad y determinar cuáles pueden ejecutarse en el entorno del navegador, contrastando los métodos agnósticos al modelo con los específicos de cada arquitectura.
3. **Implementación de explicabilidad agnóstica (SHAP):** Integrar un método basado en la perturbación de la entrada, ejecutado en cliente mediante WebSHAP, aplicable a las distintas tareas de la plataforma (clasificación y regresión de datos tabulares, clasificación de imágenes y detección de objetos).
4. **Implementación de explicabilidad específica del modelo (LRP):** Desarrollar la propagación de relevancia capa a capa sobre los modelos introspectables

entrenados en el propio navegador, así como las técnicas de segmentación en superpíxeles necesarias para las explicaciones sobre imágenes.

5. **Integración en la interfaz de usuario:** Desarrollar los componentes en React que permitan al usuario solicitar una explicación y visualizar los resultados, ya sean gráficos de importancia de características o mapas de calor sobre la imagen, de forma intuitiva.
6. **Validación funcional:** Verificar que las explicaciones generadas son coherentes con el comportamiento de los modelos ejecutados en el navegador.

1.3. Estructura de la memoria

Este documento se estructura siguiendo el ciclo de vida de un proyecto de ingeniería software:

- El **Capítulo 2** recoge la especificación del trabajo, desde la situación de partida y los antecedentes hasta las restricciones, la metodología, las tecnologías, la planificación y el presupuesto.
- El **Capítulo 3** especifica los requisitos funcionales y no funcionales del sistema, junto con sus casos de uso.
- El **Capítulo 4** aborda la arquitectura del sistema y el diseño de la solución.
- El **Capítulo 5** describe el desarrollo del módulo, organizado por incrementos, con la implementación y las pruebas de cada uno.
- Finalmente, el **Capítulo 6** expone las conclusiones y las líneas de trabajo futuro.

Capítulo 2

Especificación del trabajo

En este capítulo describimos el punto de partida del proyecto, es decir, el sistema sobre el que vamos a construir y las carencias que hemos detectado en él y que justifican el trabajo. Antes de plantear ningún requisito conviene entender bien qué ofrecía ya la plataforma *Nets4Learning* y en qué presentaba carencias, dado que esto limita el rango de las cosas que se pueden hacer en el proyecto.

2.1. Descripción de la situación de partida

El TFG parte de una herramienta desarrollada por el grupo de investigación SIMI-DAT de la Universidad de Jaén. El proyecto a realizar consiste en la ampliación de la misma, heredando por consecuente tanto sus posibilidades como sus limitaciones. Describiremos la plataforma en los siguientes apartados en su fase previa a la ampliación y tras esta.

2.1.1. Descripción del entorno actual: *Nets4Learning*

Nets4Learning es una herramienta enfocada para la enseñanza y la experimentación con modelos de aprendizaje profundo sin la necesidad de tener hardware especializado, pudiendo aprender sin tener conocimientos de programación. *Nets4Learning* propone acercar el aprendizaje automático a un público menos especializado mediante un enfoque público educativo, permitiendo a estudiantes entrenar modelos, hacer predicciones y ver cómo se comportan a través del navegador.

Como característica que define a *Nets4Learning* sería su arquitectura. Se trata de una SPA (Single Page Application) construida con REACT, con la peculiaridad de que el entrenamiento e inferencia de los modelos se hacen completamente Client-Side, de forma que se ejecuta aprovechando la GPU del dispositivo que renderiza la página. Esto permite que la herramienta funcione sin ningún tipo de infraestructura, respetando la privacidad de la información del usuario y permitiendo el acceso desde cualquier equipo moderno.

Según el tipo de problema se presentan diferentes tareas con las que interactuar, entre las que tenemos clasificación y regresión de datos tabulares, clasificación de imágenes y detección de objetos. Se hace uso de TensorFlow y diferentes modelos ya entrenados como MobileNet, además de modelos que el propio usuario puede entrenar como los de MNIST o KMNIST. La interfaz soporta varios idiomas (español, inglés y japonés).

2.1.2. Resumen de las deficiencias y carencias identificadas

La plataforma *Nets4Learning* cubre de forma decente la parte de entrenamiento y predicción. Sin embargo, aún siendo el objetivo principal de la plataforma, se podría considerar que carece de explicación o método para explicar el comportamiento de los modelos.

Especialmente en un contexto educativo, dicha carencia es realmente crítica dada la necesidad de que un estudiante comprenda de verdad como la red neuronal toma las decisiones. Conocer que características o regiones de una imagen motivan la decisión de una red neuronal permite al estudiante la interpretación de los resultados.

De dicha carencia surge la necesidad de implementar un módulo de Inteligencia Artificial Explicable (XAI) de forma que cada predicción tenga una explicación visual interpretable por el usuario. A la hora de implementar el módulo XAI se debe mantener la estructura actual de *Nets4Learning*. Al ser todo en local, no podemos hacer uso de librerías potentes ya implementadas en Python, de forma que en la siguiente sección de detallan las restricciones del proyecto.

2.2. Antecedentes

Este apartado recoge los fundamentos sobre los que se construye el proyecto. Los modelos de aprendizaje profundo presentan un problema de opacidad, para el cual utilizamos la Inteligencia Artificial Explicable. Presentaremos técnicas utilizadas en este proyecto además de analizar su rendimiento y ejecución en el navegador, entorno principal de este proyecto.

2.2.1. La Inteligencia Artificial Explicable (XAI)

El término “caja negra” se utiliza para describir aquellos modelos cuyo funcionamiento interno resulta inaccesible o incomprensible para un humano. Las redes neuronales profundas presentan millones de parámetros interconectados que hacen imposible el trabajo de análisis manual por parte de un humano.

La Inteligencia Artificial Explicable, o XAI (*Explainable Artificial Intelligence*), agrupa el conjunto de técnicas orientadas a hacer comprensibles las decisiones de estos modelos [1]. Dentro de este campo es habitual clasificar las técnicas según varios criterios. En función del alcance, se distinguen entre explicaciones *globales*, que describen el comportamiento del modelo en su conjunto, y *locales*, que son de una predicción concreta. Tenemos tanto métodos *agnósticos* como *específicos* del modelo. Los *agnósticos* tratan al modelo como una caja negra, centrándose en relacionar sus entradas con sus salidas, ignorando por completo la arquitectura interna del modelo, véase Shapley. Por otro lado, los *específicos* funcionan con base en la arquitectura interna.

2.2.2. Técnicas de explicabilidad

Métodos agnósticos: LIME y SHAP

Respecto a los métodos agnósticos, se basan en reaccionar ante cambios aplicados en la entrada. Existen dos enfoques populares, el primero de ellos sería LIME (*Local Interpretable Model-agnostic Explanations*), que popularizó el enfoque agnóstico.

SHAP (*SHapley Additive exPlanations*) está basado en los valores de Shapley de la teoría de juegos cooperativos [2], consistiendo en repartir de forma justa la contribu-

ción de cada característica a la predicción final, considerando todas las combinaciones posibles de características presentes y ausentes, es decir, comparamos excluyendo ciertas features con el fin de obtener sus contribuciones marginales. Sin embargo, presenta un coste computacional elevado dado que para una sola predicción requerimos de la evaluación del modelo durante varias iteraciones. Dicho coste computacional será el principal obstáculo al implementarlo en un entorno donde el cliente debe aportar la potencia de computación.

Métodos específicos: LRP

Frente a los anteriores, LRP (*Layer-wise Relevance Propagation*) es un método específico que sí accede al interior de la red [3]. LRP retropropaga la “relevancia” a través de cada una de las capas, hasta llegar a la entrada, permitiéndonos saber que elementos aportaron más a la predicción. Es mucho más eficiente que SHAP dado que es una sola ejecución, pero a cambio está estrechamente relacionado con la arquitectura del modelo ya que cada tipo de capa requiere su propia regla de propagación, limitando el uso a modelos cuya estructura interna sea completamente accesible.

Segmentación en superpíxeles

Cuando la entrada es una imagen, atribuir importancia a cada píxel de forma individual resulta en explicaciones ruidosas y muy costosas. Una solución habitual es agrupar previamente los píxeles en regiones coherentes, comúnmente referidas como superpíxeles, y trabajar sobre esas regiones. Para poder llevar a cabo dicha agrupación, hacemos uso del algoritmo SLIC (*Simple Linear Iterative Clustering*), que segmenta la imagen en regiones de tamaño y forma regulares en función de la proximidad y la similitud de color [4]. De este modo, las explicaciones se calculan sobre un número manejable de regiones con significado visual, en lugar de sobre miles de píxeles aislados.

2.3. Restricciones

Este proyecto presenta límites marcados por el tiempo disponible para el desarrollo y la propia plataforma *Nets4Learning*. En este apartado se recogen las principales restricciones del proyecto de implementación del módulo de explicabilidad, las cuales

explican decisiones tomadas durante el desarrollo del proyecto.

2.3.1. Restricciones temporales

EL proyecto se desarrolla con una carga de trabajo previamente definida y un plazo de entrega fijo. Se ha desarrollado a lo largo de dos cuatrimestres. La limitación temporal ha obligado a priorizar la implementación del módulo de explicabilidad en las tareas más representativas.

2.3.2. Restricción de ejecución en el cliente

La restricción más determinante del proyecto no es temporal, sino técnica, y la heredamos directamente de *Nets4Learning*. Toda la plataforma funciona en el lado del cliente, sin ningún *backend* que procese los datos, y el módulo de explicabilidad debe respetar esa misma filosofía. Esto tiene las siguientes consecuencias:

- Quedan descartadas las bibliotecas de explicabilidad más habituales y potentes, como las de Python (SHAP, LIME, etc) que requieren de un servidor para funcionar en la Web. Nos obligamos, por tanto, a apoyarnos en soluciones nativas del navegador, como WebSHAP y TensorFlow.js.
- El cómputo depende de la GPU del propio equipo del usuario a través de WebGL, con unos límites de memoria y de rendimiento mucho más estrictos que los de un servidor.
- Si bien las restricciones presentan varios problemas, por otro lado nos presentan dos ventajas principales: la privacidad (los datos del usuario nunca salen de su equipo) y la accesibilidad (no hay que instalar nada ni disponer de hardware especializado).

2.3.3. Restricciones de integración

Al tratarse de una ampliación y no de un sistema nuevo, el módulo debe integrarse en la arquitectura existente de *Nets4Learning* sin romper lo que ya funcionaba. Esto implica respetar decisiones estructurales de la plataforma, además de adaptar todo al soporte multilingüe que ya incorporaba.

2.4. Metodología de desarrollo software

Para el desarrollo del proyecto se debe decidir que metodología aplicar. De entre las varias metodologías de desarrollo software que se han planteado para el desarrollo del proyecto, expondremos las más habituales, contando sus inconvenientes y sus ventajas. Se justificará la elección final y que se ha sopesado en cada una de las metodologías contempladas.

- **Modelo en cascada.** Plantea el desarrollo como una sucesión de fases que se recorren una detrás de otra. El problema principal es que cualquier ajuste o cambio realizado obliga a rehacer una gran parte de lo anteriormente desarrollado.
- **Modelos iterativos e incrementales.** En lugar de entregarlo todo al final, se van completando incrementos, y cada uno aporta una porción funcional del sistema que ya se puede probar y validar por separado.
- **Metodologías ágiles.** Llevan esa idea un paso más allá con iteraciones cortas y una adaptación continua a los cambios y a la realimentación, siendo Scrum una de las más conocidas. Esta fue una candidata muy prometedora.

De entre todas, hemos optado por un **desarrollo incremental**. En este modelo el producto no se construye de una sola vez, este se hace de forma progresiva iterando versiones denominadas incrementos. Es importante entender que cada incremento debe representar una parte del sistema que sea funcional y que puede entregarse sin el resto de incrementos. De esta forma, el proyecto se reduce a piezas más pequeñas que se desarrollan en orden. La Figura [2.1](#) ilustra esta idea aplicada a nuestro caso, en el que cada incremento se corresponde con uno de los módulos de explicabilidad.

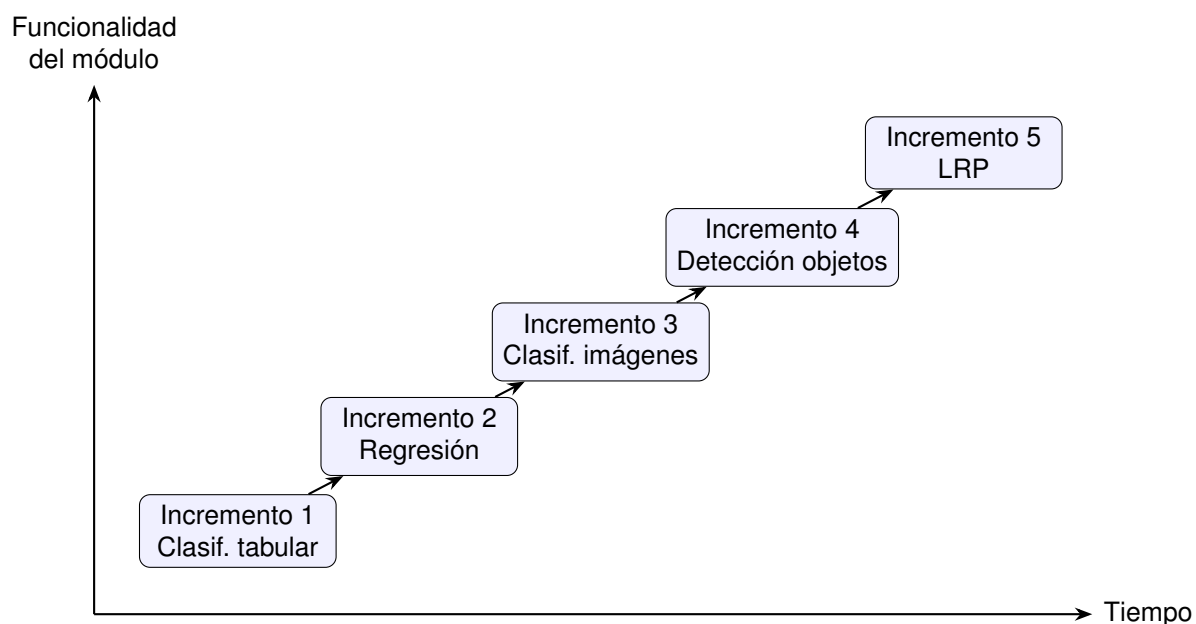


Figura 2.1: Modelo incremental aplicado al proyecto. Cada incremento añade la explicabilidad de una tarea y, dentro de él, se recorren las fases de análisis, diseño, implementación y pruebas.

En nuestro caso, dadas las siguientes razones hemos elegido hacer uso de la metodología incremental.

- **Descomposición natural.** El módulo de explicabilidad ya estaba formado, de por sí, por submódulos independientes —uno por cada tarea de la plataforma—, y encajan como incrementos sin tener que forzar ninguna división artificial.
- **Entrega y validación tempranas.** Como cada incremento es funcional por sí mismo, podemos probarlo y validarlo en cuanto está listo, sin necesidad de esperar a que el proyecto entero esté terminado.
- **Reducción de riesgos.** Al ir entregando en partes pequeñas, los problemas se detectan y se corrigen en etapas tempranas, cuando arreglarlos es mucho menos costoso que al final.
- **Flexibilidad ante los cambios.** Cualquier ajuste que surja se puede incorporar en los incrementos siguientes con poco coste, sin obligarnos a rehacer lo que ya estaba hecho.
- **Reflejo del desarrollo real.** Por último, esta forma de trabajar coincide con cómo se construyó de verdad el módulo, porque fuimos incorporando la explicabilidad tarea a tarea a lo largo de los dos cuatrimestres, empezando por los datos tabulares y terminando con LRP.

Sobre esta base incremental nos hemos apoyado, además, en prácticas habituales del desarrollo ágil, como el control de versiones con Git y las revisiones periódicas con los tutores. Más adelante, en el capítulo de desarrollo, describimos cada uno de estos incrementos por separado.

2.5. Estimación del tamaño y el esfuerzo

La estimación del tamaño y el esfuerzo de este proyecto se apoya en dos enfoques complementarios, las líneas de código (LOC) para el tamaño y el modelo COCOMO para el esfuerzo. Conviene tener presente que se trata de una arquitectura que funciona por completo en el lado del cliente, así que no hay componentes de servidor ni persistencia en base de datos que estimar.

2.5.1. Estimación del tamaño: líneas de código (LOC)

Para estimar el tamaño partimos del código realmente escrito para el módulo de explicabilidad, agrupado por bloques funcionales. La Tabla 2.1 recoge una medida aproximada de cada uno de ellos.

Bloque funcional	LOC aprox.
Muestreo y preparación de datos para SHAP	100
Funciones predictoras y adaptadores de los modelos	420
Segmentación en superpíxeles (SLIC, SLIC0 y segmentación facial)	1230
Propagación de relevancia (LRP) y captura de activaciones	600
Orquestación de la explicación por cada tarea	430
Visualización (mapas de calor, gráficos de importancia y diagrama de enjambre)	440
Total en módulos dedicados	≈ 3220

Tabla. 2.1: Tamaño aproximado del módulo de explicabilidad, en líneas de código.

A esas cifras hay que sumar la integración del módulo en la interfaz ya existente de *Nets4Learning* (controles para lanzar las explicaciones, paneles de resultados y

adaptación multilingüe), que está repartida por las distintas páginas de la plataforma, con lo que el tamaño total del módulo ronda las 3,5 KLOC.

Ahora bien, para estimar el esfuerzo no tiene sentido tratar todo ese tamaño como código escrito desde cero, porque no lo es. Por un lado, las bibliotecas externas en las que nos apoyamos (WebSHAP y TensorFlow.js) son dependencias que no hemos programado nosotros y, por tanto, no cuentan. Por otro lado, la segmentación en superpíxeles (unas 1,1 KLOC entre SLIC y SLIC0) parte de implementaciones previas que únicamente hemos adaptado. Descontando esa parte reutilizada, el código realmente desarrollado en este trabajo ronda las 2,1 **KLOC**, que es la cifra sobre la que tiene sentido aplicar el modelo de esfuerzo.

2.5.2. Estimación del esfuerzo: modelo COCOMO

Para estimar el esfuerzo utilizamos el modelo COCOMO en su versión básica, que relaciona el tamaño del software con el esfuerzo y la duración mediante un par de expresiones. Dado que se trata de un proyecto pequeño, desarrollado por una sola persona y sin requisitos especialmente rígidos, encaja en el modo **orgánico**, cuyas fórmulas son:

$$E = 2,4 \cdot (\text{KLOC})^{1,05} \quad T = 2,5 \cdot E^{0,38}$$

donde E es el esfuerzo en personas-mes, T la duración en meses y KLOC los miles de líneas de código. Sustituyendo el tamaño del código realmente desarrollado (2,1 KLOC) obtenemos los valores de la Tabla 2.2.

Magnitud	Valor
Esfuerzo estimado (E)	$\approx 5,2$ personas-mes
Duración estimada (T)	$\approx 4,7$ meses
Personal medio (E/T)	$\approx 1,1$ personas

Tabla. 2.2: Estimación del esfuerzo mediante COCOMO básico (modo orgánico).

Estas cifras deben interpretarse con cierta cautela. Un esfuerzo de $\approx 5,2$ personas-mes equivale, a razón de 152 horas por persona-mes, a unas 795 horas de trabajo, que siguen estando por encima de las aproximadamente 300 horas de dedicación que corresponden a los 12 ECTS del TFG. Esta diferencia es esperable, porque COCOMO

está calibrado para desarrollo profesional a tiempo completo y tiende a sobreestimar los proyectos pequeños llevados por una sola persona. Aun así, la estimación resulta útil, porque confirma que el código desarrollado tiene entidad suficiente para un Trabajo Fin de Grado, y el hecho de haberlo podido completar dentro del calendario de dos cuatrimestres se explica, además, por el apoyo en la base ya existente de *Nets4Learning* y en herramientas como WebSHAP o TensorFlow.js.

2.6. Tecnologías utilizadas

En el desarrollo de este proyecto, a la hora de elegir tecnologías debíamos tener en cuenta las tecnologías ya implementadas. Dado que se trata de una ampliación de *Nets4Learning*, el trabajo se hace sobre las tecnologías ya existentes. Estas son tecnologías web que permiten ejecutar modelos directamente en el lado del cliente, y por supuesto, es tanto la filosofía del proyecto como el elemento más importante a la hora de estudiar las tecnologías a implementar por encima del proyecto.

- **React.** Biblioteca de JavaScript sobre la que está construida la interfaz de *Nets4Learning*, organizada como una aplicación web de página única (SPA). El módulo de explicabilidad se implementa en React como componentes de interfaz.
- **JavaScript y TypeScript.** Estos lenguajes son los usados en el proyecto. Tras una actualización de la plataforma se hace el cambio a TypeScript donde se tiene un tipado especial que permite mantener código más robusto y de mayor calidad.
- **TensorFlow.js.** Es el elemento más importante de todo el proyecto, dado que permite la implementación de modelos de aprendizaje automático para uso en navegador sin necesidad de backend.
- **WebGL.** Tecnología web que TensorFlow.js utiliza por debajo para acelerar los cálculos en la GPU del cliente.
- **WebSHAP.** Implementación de SHAP escrita en JavaScript y TypeScript y pensada para ejecutarse por completo en el navegador. Gracias a WebSHAP no requerimos de Python ni implementaciones complejas para poder usar Shapley Additive Values en el módulo de explicabilidad.

2.6.1. Ejecución de modelos en el navegador

Como se menciona en numerosas ocasiones a lo largo de esta memoria, la ejecución de modelos en el navegador es la filosofía de este proyecto. Sin embargo, históricamente se asocia el aprendizaje automático a Python, de modo que con la aparición de TensorFlow.js se cambia el paradigma de forma drástica permitiendo entrenar y ejecutar modelos en el navegador.

Las bibliotecas de XAI más extendidas están escritas en Python y dan por hecho la presencia de un *backend*, así que no se pueden utilizarse directamente en el navegador. Esto es la mayor ventaja y limitación que tiene el proyecto actualmente. Sin embargo, dicha limitación puede superarse con el uso de WebSHAP, la cual es una implementación de SHAP hecha para ser ejecutada en el lado del cliente mediante TensorFlow.js, asegurando la privacidad de los datos.

A pesar de todo, el panorama de los modelos en el navegador sigue cambiando, y recientemente se ha empezado a extender el uso de WebGPU, que permite trabajar con modelos mucho más grandes mediante nuevos motores como ONNX Runtime Web o Transformers.js.

2.7. Planificación

El proyecto se ha desarrollado a lo largo de dos cuatrimestres. El primero, más denso, concentró la investigación inicial y los primeros incrementos de explicabilidad basada en SHAP (datos tabulares, regresión, imágenes y detección de objetos), mientras que el segundo se dedicó sobre todo a la explicabilidad específica mediante LRP y a mejorar la aplicación base. La redacción de la memoria se ha llevado a cabo de forma transversal durante todo el periodo. Para la gestión y el desarrollo nos hemos apoyado en herramientas de código abierto o gratuitas, principalmente Git y GitHub para el control de versiones, SourceTree como cliente visual, Visual Studio Code como entorno de desarrollo y $\text{\LaTeX}$ (a través de Overleaf) para la redacción de la memoria.

La Tabla 2.3 resume las fases e incrementos en que se ha dividido el proyecto, con su duración aproximada, las actividades principales de cada uno y los resultados obtenidos.

Fase / Incremento	Duración	Actividades principales	Resultados
Investigación y formación	Meses 1–2	Estudio de la XAI y de las técnicas SHAP y LRP; familiarización con React y TensorFlow.js	Base teórica y dominio del entorno de trabajo
Análisis del sistema existente	Mes 2	Estudio del código de <i>Nets4Learning</i> y de cómo se utilizan sus modelos	Comprensión necesaria para integrar la explicabilidad
Incremento 1: clasificación tabular	Mes 3	Integración de WebSHAP y explicación por perturbaciones sobre datos tabulares	Explicabilidad SHAP en clasificación tabular
Incremento 2: regresión	Meses 3–4	Extensión de la explicabilidad tabular a los modelos de regresión	Explicabilidad en regresión
Incremento 3: clasificación de imágenes	Meses 4–5	Segmentación en superpíxeles (SLIC) y visualización mediante mapas de calor	Explicabilidad SHAP sobre imágenes
Incremento 4: detección de objetos	Meses 5–6	Segmentación facial y por rejilla (SLIC0) y enmascarado por desenfoque	Explicabilidad en detección de objetos
Incremento 5: LRP	Meses 6–8	Propagación de relevancia capa a capa y captura de activaciones	Explicabilidad específica sobre modelos introspectables
Pruebas y validación	Meses 8–9	Pruebas unitarias y validación funcional de cada incremento	Módulo validado
Redacción de la memoria	Transversal	Documentación continua del trabajo	Memoria completa

Tabla. 2.3: Planificación temporal del proyecto por fases e incrementos.

Partiendo de esta tabla, y con una dedicación relativamente constante, se aprecia que cada mes se ha centrado en una fase o en un incremento concreto. En primer lugar se debía investigar y analizar el funcionamiento de sistema. A continuación tocaban los incrementos de SHAP, por el que comenzamos a desarrollarlo para la clasificación tabular siguiendo con las imágenes y la detección de objetos. El segundo cuatrimestre se centra de forma exclusiva en la implementación de LRP y su lógica subyacente dado que presenta una complejidad mayor y debe ser tratado como tal. Por otro lado, la memoria se escribe a lo largo de todo el tiempo de desarrollo. Esta distribución se resume en el diagrama de Gantt de la Figura 2.2.

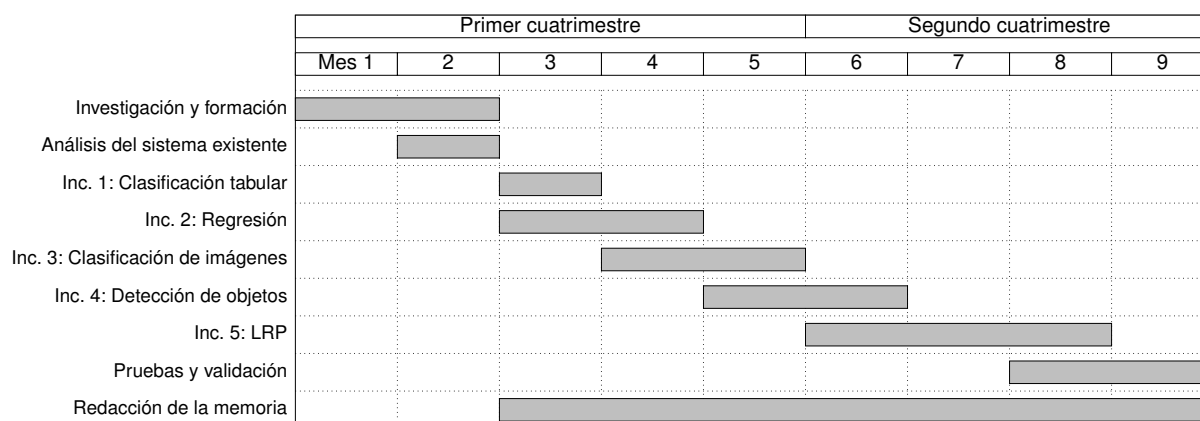


Figura 2.2: Diagrama de Gantt con la planificación temporal del proyecto a lo largo de los dos cuatrimestres.

2.8. Presupuesto

A lo largo del desarrollo del proyecto no se hace uso de herramientas de pago, sino únicamente de herramientas y bibliotecas de código abierto, con lo que el coste en licencias de software es nulo. El presupuesto se concentra, por tanto, en los costes de personal y en el equipo informático necesario para el desarrollo.

2.8.1. Costes de personal

Se estima la dedicación del proyecto en torno a las horas asociadas a la carga en créditos ECTS del TFG.

Rol	Horas	Coste/hora	Total
Desarrollador (estudiante)	360	18,00 €	6.480,00 €
Supervisión (tutores)	30	40,00 €	1.200,00 €

Tabla. 2.4: Estimación de costes de personal.

2.8.2. Costes de hardware y software

Como recurso material se hace uso de un portátil donde se desarrolla el proyecto. La Tabla 2.5 recoge estos costes.

Concepto	Descripción	Total
Equipo informático	Amortización del portátil de desarrollo durante el periodo del proyecto	150,00 €
Software	Herramientas de código abierto y gratuitas	0,00 €

Tabla. 2.5: Estimación de costes de hardware y software.

2.8.3. Coste total

Sumando los conceptos anteriores, el coste total estimado del proyecto asciende a **7.830,00 €**, tal y como se refleja en la Tabla 2.6.

Concepto	Total
Costes de personal	7.680,00 €
Costes de hardware y software	150,00 €
Total	7.830,00 €

Tabla. 2.6: Coste total estimado del proyecto.

Capítulo 3

Análisis de requisitos

En este capítulo especificamos los requisitos del módulo de explicabilidad. Hay que tener en cuenta que nuestro sistema no es una aplicación independiente, sino una ampliación de la plataforma *Nets4Learning*, y por eso los requisitos los hemos definido pensando tanto en la funcionalidad que debe aportar el nuevo módulo como en las restricciones que impone integrarse en un sistema que ya existe. A continuación los recogemos primero como requisitos funcionales y no funcionales, y después los concretamos en sus correspondientes casos de uso. Las necesidades concretas de cada tarea, en forma de historias de usuario, se detallan más adelante dentro de cada incremento del capítulo de desarrollo.

3.1. Requisitos funcionales

Los requisitos funcionales recogen las capacidades concretas que el módulo tiene que ofrecer, y los resumimos en la Tabla [3.1](#).

3.2. Requisitos no funcionales

Los requisitos no funcionales, por su parte, establecen las condiciones de calidad y las restricciones bajo las que tiene que funcionar el sistema, y los recogemos en la Tabla [3.2](#).

ID	Descripción
RF-1	El usuario podrá solicitar una explicación de la predicción realizada por un modelo previamente entrenado o cargado.
RF-2	El sistema generará explicaciones mediante un método agnóstico al modelo (SHAP) para las tareas de clasificación y regresión de datos tabulares, clasificación de imágenes y detección de objetos.
RF-3	El sistema generará explicaciones mediante un método específico del modelo (LRP) para los modelos de imagen cuya arquitectura sea accesible.
RF-4	En las tareas de imagen, el sistema segmentará la imagen en superpíxeles antes de calcular la explicación.
RF-5	El sistema presentará las explicaciones de forma visual, mediante gráficos de importancia de características para datos tabulares y mapas de calor superpuestos para imágenes.
RF-6	El usuario podrá ajustar los parámetros de la explicación, como el número de muestras empleadas por SHAP o la granularidad de la segmentación.
RF-7	El sistema deshabilitará automáticamente los métodos de explicación no aplicables al modelo seleccionado.
RF-8	La explicación se calculará sobre la clase predicha o sobre las etiquetas seleccionadas por el usuario.

Tabla. 3.1: Requisitos funcionales del sistema.

3.3. Casos de uso

El actor principal del sistema es el **usuario** de la plataforma que normalmente será un estudiante que quiere entender cómo se comporta un modelo y sus predicciones. En la Figura 3.1 mostramos el diagrama de casos de uso del módulo y en las tablas siguientes vamos detallando cada uno de ellos.

ID	Descripción
RNF-1	Ejecución en cliente. Todo el cálculo de las explicaciones deberá realizarse en el navegador, sin depender de ningún servidor que procese los datos.
RNF-2	Integración. El módulo deberá integrarse en la arquitectura existente de <i>Nets4Learning</i> (React y TensorFlow.js) sin alterar el funcionamiento de las funcionalidades previas.
RNF-3	Usabilidad. La interfaz deberá ser intuitiva y adecuada para un público educativo sin conocimientos avanzados de aprendizaje automático.
RNF-4	Rendimiento. Las explicaciones deberán generarse en un tiempo razonable dentro de las limitaciones del entorno del navegador, ofreciendo al usuario información sobre el progreso del cálculo.
RNF-5	Extensibilidad. El diseño del módulo deberá estar desacoplado de los modelos concretos de modo que puedan incorporarse nuevos modelos o técnicas con un impacto mínimo.
RNF-6	Internacionalización. El módulo deberá respetar el soporte multilingüe de la plataforma (español, inglés y japonés).

Tabla. 3.2: Requisitos no funcionales del sistema.

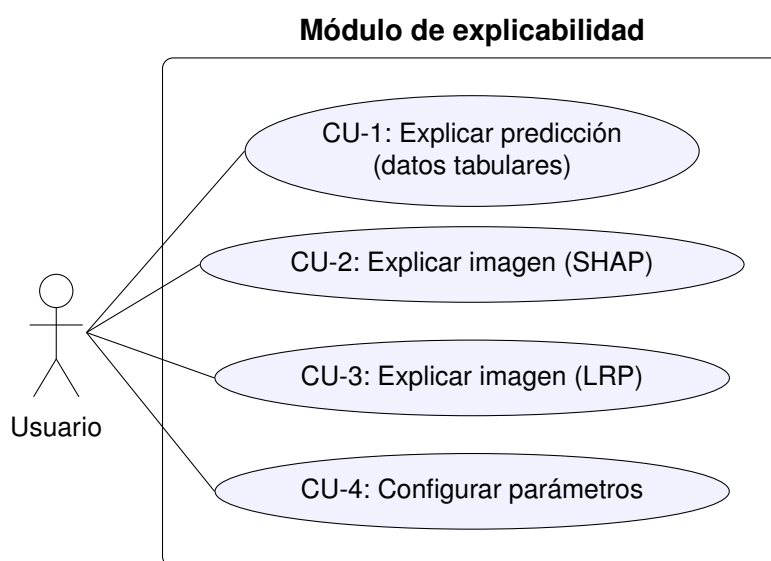


Figura 3.1: Diagrama de casos de uso del módulo de explicabilidad.

Campo	Descripción
Identificador	CU-1
Nombre	Explicar una predicción sobre datos tabulares
Actor	Usuario
Precondición	Existe un modelo tabular entrenado y se ha realizado una predicción sobre una instancia.
Flujo principal	1. El usuario solicita la explicación de la predicción. 2. El sistema calcula las contribuciones de cada característica mediante SHAP, en el navegador. 3. El sistema muestra un gráfico de importancia de características.
Postcondición	El usuario visualiza qué atributos han influido más en la predicción y en qué sentido.

Tabla. 3.3: Caso de uso CU-1: explicación sobre datos tabulares.

Campo	Descripción
Identificador	CU-2
Nombre	Explicar una predicción sobre una imagen (SHAP)
Actor	Usuario
Precondición	Existe un modelo de imagen y se ha clasificado una imagen de entrada.
Flujo principal	1. El usuario solicita la explicación. 2. El sistema segmenta la imagen en superpíxeles. 3. El sistema evalúa el modelo sobre versiones perturbadas de la imagen y calcula las contribuciones mediante SHAP. 4. El sistema muestra un mapa de calor superpuesto a la imagen.
Postcondición	El usuario visualiza qué regiones de la imagen han sido determinantes para la predicción.

Tabla. 3.4: Caso de uso CU-2: explicación de imagen mediante SHAP.

Campo	Descripción
Identificador	CU-3
Nombre	Explicar una predicción sobre una imagen (LRP)
Actor	Usuario
Precondición	Existe un modelo de imagen de arquitectura accesible (entrenado en la plataforma) y se ha clasificado una imagen.
Flujo principal	1. El usuario selecciona el método LRP y solicita la explicación. 2. El sistema propaga la relevancia desde la salida hasta la entrada, capa a capa. 3. El sistema muestra un mapa de calor con la relevancia obtenida.
Postcondición	El usuario visualiza la relevancia que el modelo atribuye a cada región de la imagen.
Excepción	Si el modelo no es introspectable, el método LRP no está disponible (véase RF-7).

Tabla. 3.5: Caso de uso CU-3: explicación de imagen mediante LRP.

Campo	Descripción
Identificador	CU-4
Nombre	Configurar los parámetros de la explicación
Actor	Usuario
Precondición	El usuario ha seleccionado un método de explicación.
Flujo principal	1. El usuario ajusta los parámetros disponibles (número de muestras, granularidad de la segmentación, etc.). 2. El sistema emplea esos valores en el cálculo de la siguiente explicación.
Postcondición	La explicación se genera con la configuración indicada por el usuario.

Tabla. 3.6: Caso de uso CU-4: configuración de los parámetros de la explicación.

Capítulo 4

Diseño del sistema

En este capítulo describimos el diseño de la solución que hemos desarrollado. En primer lugar planteamos la arquitectura y de cómo el nuevo módulo de explicabilidad encaja dentro de la plataforma, para después ir bajando al diseño interno del propio módulo. A lo largo del capítulo iremos justificando las decisiones que hemos ido tomando, haciendo énfasis en las decisiones que se ven condicionadas por la restricción de tener que ejecutarlo todo en el lado del cliente (*client-side*), dado que ha sido lo que más ha condicionado el desarrollo y elección de herramientas.

4.1. Arquitectura general

Nets4Learning es una aplicación web de página única (*SPA*) construida con React, en la que la inferencia de los modelos se realiza en el lado del cliente haciendo uso de TensorFlow.js. El módulo de explicabilidad se integra como una capa más dentro de esa misma arquitectura y dado que va a formar parte de la experiencia final del usuario, le damos la misma importancia que al resto de la interfaz.

El sistema se organiza en tres niveles:

- **Capa de modelos:** los modelos de aprendizaje automático que ya ofrecía la plataforma, agrupados por tipo de tarea (clasificación y regresión sobre datos tabulares, clasificación de imágenes y detección de objetos).
- **Capa de explicabilidad:** el núcleo desarrollado en este trabajo, que se encarga de generar las explicaciones a partir de las predicciones de los modelos. Es independiente de los modelos concretos.

- **Capa de presentación:** ciertos componentes de la interfaz son añadidos para poder hacer funcional el módulo de explicabilidad.

La Figura 4.1 muestra esta organización y las relaciones entre las tres capas.

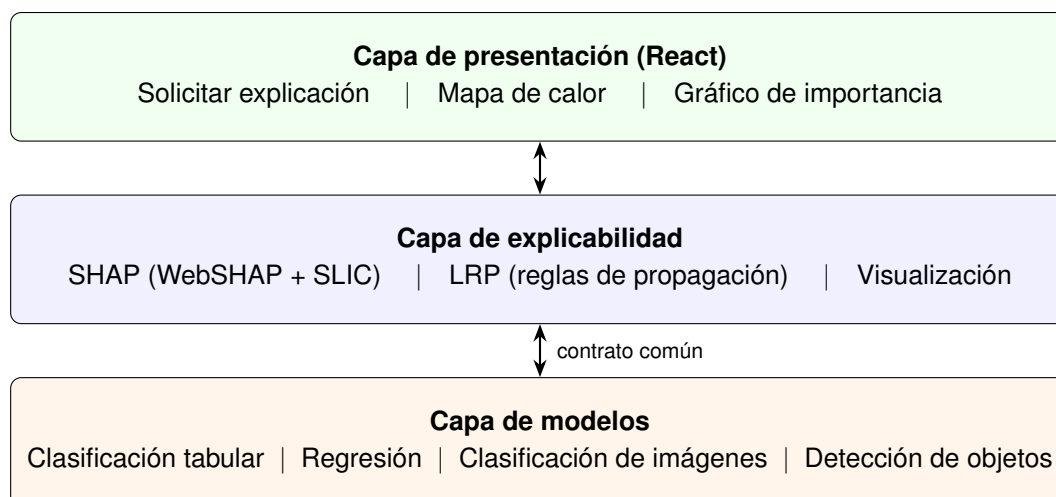


Figura 4.1: Arquitectura general del sistema con la capa de explicabilidad integrada.

4.2. Integración con Nets4Learning

Probablemente la decisión de diseño más importante de todo el proyecto es la forma en que la capa de explicabilidad se comunica con los modelos ya que de ella depende que el módulo se pueda reutilizar. Con el fin de poder aplicarlo tanto en la clasificación de imágenes como en los datos tabulares sin tener que conocer los detalles internos de cada modelo, hacemos uso de interfaces comunes de forma que la explicabilidad no trabaja contra un modelo concreto, sino contra un contrato que todos ellos cumplen.

Lo que hacemos es que cada tipo de tarea cuente con una interfaz común que implementan todos los modelos de esa tarea y en la que se declaran, entre otras, las operaciones necesarias para habilitar un modelo y para obtener una predicción a partir de una entrada.

Este enfoque se corresponde con el patrón de diseño *Template*, en el que la explicabilidad trabaja siempre contra la misma abstracción mientras que cada modelo concreto aporta su propia implementación, dado que entre unos y otros hay bastantes diferencias. Gracias a ello cumplimos el requisito de extensibilidad (RNF-5) dado que añadir un nuevo modelo no requiere más cambios que la adaptación a la interfaz.

4.3. Diseño del módulo de explicabilidad

Dentro del módulo tenemos dos técnicas que, aunque las dos sirven para explicar, no se solapan entre sí dado que por un lado tenemos el método agnóstico de caja negra (SHAP) y, por otro, el método específico (LRP). Si bien son idénticos a nivel de interfaz, su estructura interna es completamente diferente, por lo que conviene tratarlas como tal.

4.3.1. Explicabilidad agnóstica mediante SHAP

Como el método agnóstico trata al modelo como una caja negra y lo único que necesita es poder evaluarlo una y otra vez, su diseño nos queda organizado en torno a tres elementos:

- **Función predictora.** Una función que, dada una o varias entradas, devuelve las predicciones del modelo invocando la operación de predicción de la interfaz. Actúa como adaptador entre el formato que espera el algoritmo de SHAP y el que ofrece el modelo, por lo que su diseño sigue el patrón *Adapter*.
- **Algoritmo de SHAP.** Se emplea una implementación de KernelSHAP capaz de ejecutarse en el navegador, que recibe la función predictora y un conjunto de datos de referencia, y devuelve la contribución de cada característica a la predicción.
- **Segmentación en superpíxeles.** En las tareas de imagen, antes de aplicar SHAP la imagen se divide en superpíxeles mediante el algoritmo SLIC. De este modo, las características sobre las que opera SHAP no son píxeles individuales, sino regiones con significado visual, lo que reduce drásticamente el número de evaluaciones necesarias y mejora la interpretabilidad del resultado.

El flujo de generación de una explicación mediante SHAP para una imagen se presenta en el diagrama de secuencia de la Figura 4.2, donde se segmenta la imagen, se generan versiones perturbadas activando y desactivando regiones, se evalúa el modelo sobre cada una y a partir de cómo cambia la predicción se estima la contribución de cada región.

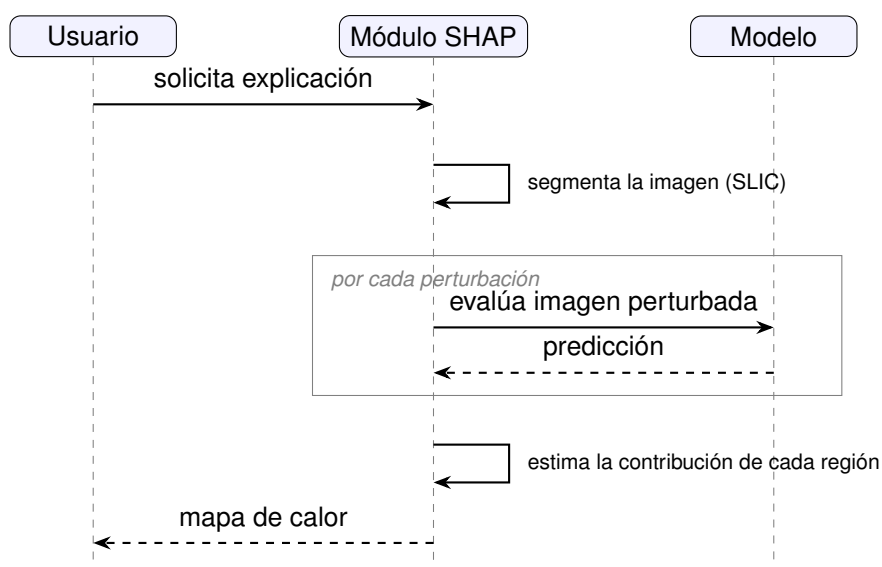


Figura 4.2: Diagrama de secuencia de la generación de una explicación mediante SHAP.

4.3.2. Explicabilidad específica mediante LRP

A diferencia de SHAP, el método LRP necesita acceder al interior de la red, por lo que su diseño nos obliga a ampliar el contrato de los modelos introspectables con dos operaciones más, una para obtener las activaciones de las capas durante una inferencia y otra para llevar a cabo la propagación de la relevancia.

De forma práctica, la propagación funciona como un recorrido de la red en sentido inverso de forma que, partiendo de la salida, vamos repartiendo la relevancia hacia las capas anteriores hasta llegar a la entrada. Como cada tipo de capa necesita una regla de propagación distinta, el diseño contempla diferentes reglas que se van aplicando según el tipo de la capa que estemos procesando en cada paso.

Sin embargo, dado que LRP depende por completo de la arquitectura, solo lo ofrecemos para los modelos cuya estructura es accesible, es decir, para el resto la interfaz simplemente no implementa las operaciones de LRP y el sistema deshabilita esta opción de forma automática (RF-7).

4.4. Diseño de la visualización

En líneas generales, tanto una técnica como la otra nos devuelven básicamente lo mismo, es decir, un valor de importancia o de relevancia asociado a cada característica

o región de la entrada (en función de si se trata de una imagen o características). A partir de ahí, el componente de visualización se encarga de convertir esos valores en algo que el usuario pueda entender fácilmente:

- **Datos tabulares o Regresión:** la importancia de cada característica se representa mediante un gráfico de barras, que indica tanto la magnitud como el sentido de la contribución de cada atributo.
- **Imágenes:** los valores de importancia se proyectan sobre la imagen original en forma de mapa de calor, asociando a cada región un color en función de su contribución. De este modo el usuario percibe de un vistazo en qué zonas se ha fijado el modelo.

El diseño de este componente funciona de forma independiente del método que haya generado los valores de manera que tanto las explicaciones de SHAP como las de LRP comparten la misma lógica de visualización en el caso de las imágenes.

4.5. Patrones de diseño aplicados

Esta Tabla 4.1 recoge los principales patrones de diseño empleados en la solución y el problema que resuelven.

Patrón	Uso en el sistema
Strategy / Template Method	Contrato común de los modelos y selección de la regla de propagación de LRP según el tipo de capa.
Adapter	Función predictora que adapta la interfaz de los modelos al formato esperado por el algoritmo de SHAP.
Factory	Creación de la función predictora adecuada para cada tipo de tarea.

Tabla. 4.1: Patrones de diseño aplicados en la solución.

Capítulo 5

Desarrollo

En este capítulo describimos el desarrollo del módulo de explicabilidad. Siguiendo la metodología incremental que justificamos en la Sección 2.4, el trabajo se organiza en cinco incrementos, uno por cada tarea a la que damos explicabilidad. Antes de entrar en ellos recogemos los fundamentos técnicos comunes sobre los que se apoyan todos. Para cada incremento seguimos después la misma estructura, con su objetivo y las historias de usuario que cubre, el diseño detallado, la implementación y el plan de pruebas.

5.1. Fundamentos comunes

Antes de abordar los incrementos conviene describir las piezas técnicas que comparten todos, porque se reutilizan una y otra vez a lo largo del desarrollo.

5.1.1. Bibliotecas empleadas: WebSHAP y SLIC

A la hora de implementar una adaptación del algoritmos SHAP basado en la teoría de juegos, hacemos uso de WebSHAP, que fue publicado por Zijie J. Wang y Duen Horng Chau, ambos miembros del Instituto de tecnología de Georgia [5]. WebSHAP hace uso de Javascript y Typescript, permitiendo su ejecución de forma local y sin necesidad de backend. Por otro lado, también va acompañado de una implementación del algoritmo de segmentación SLIC para Javascript, el cuál ha sido integrado y adaptado al proyecto.

SLIC funciona como una adaptación mejorada de K-Means, el algoritmo que divide un conjunto de datos en k grupos o *clusters* de forma que los puntos de un mismo grupo se parezcan más entre sí que a los de otros grupos. Partimos de la implementación incluida en WebSHAP [5, 6], cuya estructura se refleja en el diagrama de clases de la Figura 5.1, sobre la que se debió de realizar los ajustes necesarios para integrarla en el flujo de TensorFlow.js de la plataforma.

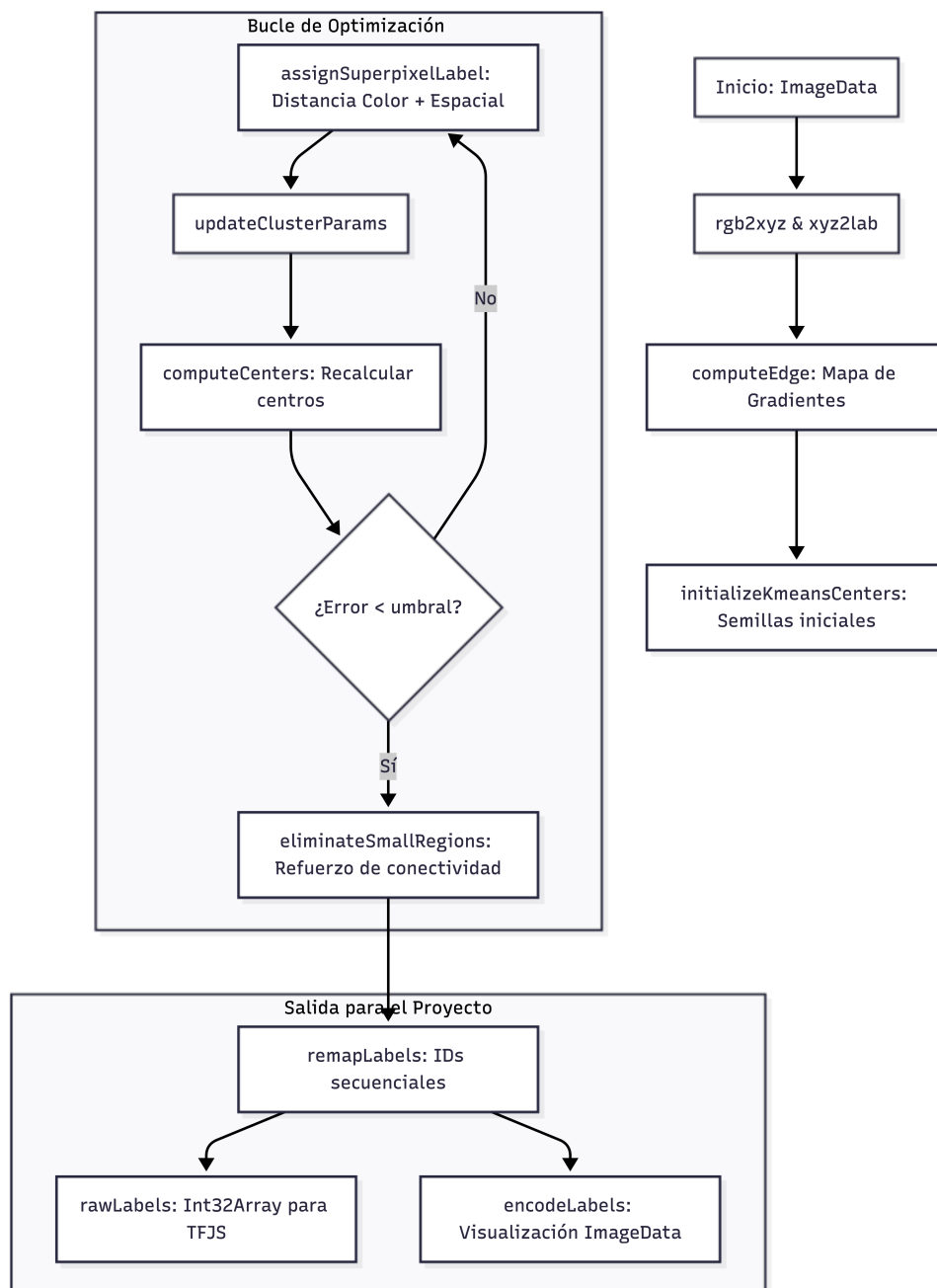


Figura 5.1: Diagrama de clases del algoritmo SLIC.

5.1.2. Mecanismo general de SHAP

La clave de SHAP es que es un método agnóstico, por lo que es aplicable a prácticamente todas las tareas que ofrece la plataforma. KernelSHAP trata el modelo como una caja negra y lo evalúa calculando la contribución marginal de cada una de las características.

Para el correcto funcionamiento de SHAP en todas las tareas, se crea una función predictora, a la cual se le pasa un conjunto de entradas que debe adaptar para poder obtener predicciones en función del modelo a evaluar. Se invoca la predicción las veces necesarias hasta obtener el resultado, de esta forma podemos hacer que el algoritmo se adapte a cualquier modelo. Por dentro el cálculo es el mismo en esencia, a excepción de algunos modelos que tratan imágenes y estas requieren un tratamiento especial aunque sea esencialmente el mismo.

Cálculo de una explicación con KernelSHAP

```
1 // Función predictora que adapta el modelo a KernelSHAP
2 const predictor = imageClassificationWrapper(
3   iModel, modelInstance, imageData, segmentationTensor, /* ... */
4 );
5
6 // Datos de referencia (ausencia de información) y explicador
7 const backgroundData = buildZeroBackground(numSegments);
8 const explainer = new KernelSHAP(predictor, backgroundData, 0.2022)
9   ;
10
11 // Cálculo de las contribuciones para la instancia de entrada
12 const shapValues = await explainer.explainOneInstance(inputVector,
13   nSamples);
```

El parámetro `nSamples` controla cuántas combinaciones de regiones evalúa el algoritmo. A mayor número de combinaciones, más precisa es la explicación, pero supone un alto coste de computación, así que es uno de los parámetros que el usuario puede ajustar (RF-6) para buscar el equilibrio entre precisión y velocidad más adecuado a su caso.

5.2. Incremento 1: explicabilidad en clasificación tabular

5.2.1. Objetivo del incremento

El primer incremento trata la tarea más sencilla de adaptar de todas, siendo esta la clasificación de datos tabulares. Cubre las siguientes historias de usuario:

- **HU-1:** como estudiante, quiero solicitar una explicación de una predicción, para entender por qué el modelo ha tomado esa decisión.
- **HU-2:** como estudiante, quiero ver qué características influyen más en una predicción sobre datos tabulares, para identificar los atributos más relevantes.

5.2.2. Diseño detallado

La función predictora tabular es la más simple de todas porque cada característica de la instancia es por sí misma válida como una entrada del SHAP. La Figura 5.2 recoge el diagrama de clases del incremento, con los nombres reales de la implementación. Se crea la función predictora mediante `myModelWrapper` y se entrega a `KernelSHAP`, que la va invocando perturbando los datos (es decir, modificar características o apagarlas), a su vez, consulta al modelo a través del contrato común. Cuando el cálculo termina, recibe los valores SHAP y se los pasa a los componentes de visualización.

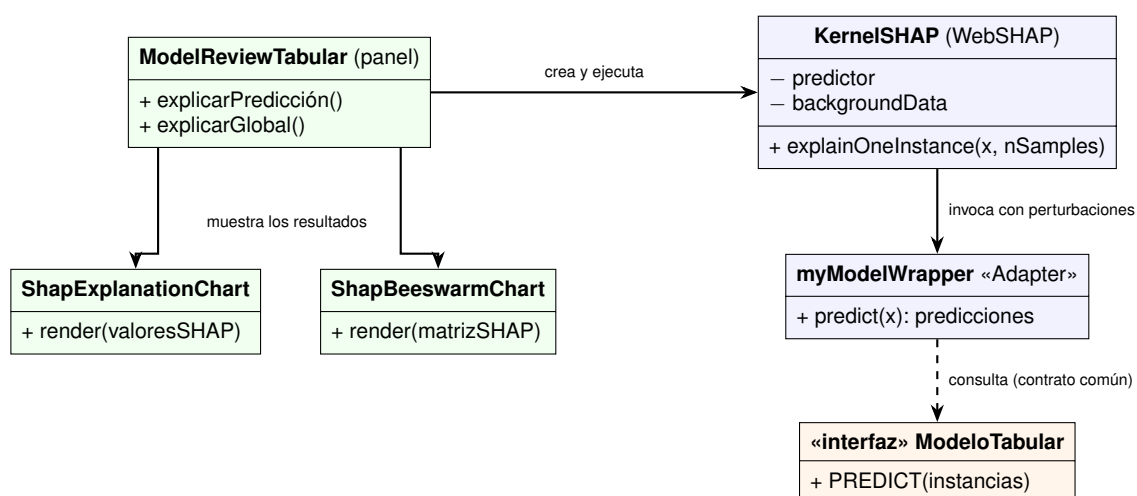


Figura 5.2: Diagrama de clases del incremento de clasificación tabular.

A la hora de implementar la interfaz se pretende integrarlo con la existente en la plataforma, por lo que simplemente ponemos la sección de explicabilidad debajo de la de predicción, de esta forma, el usuario entiende que tras la predicción tiene acceso a la explicabilidad, es importante entender dicha estructura. Aquí podemos ver el mockup donde se ve cuál es la idea principal tras la interfaz. A la hora de elegir la explicación, como tenemos dos opciones posibles que son la global o local, se muestra un gráfico de barras en local pero a la hora de explicar globalmente el modelo podemos ver que se puede elegir entre un gráfico de barras o mostrarlo en forma de diagrama de enjambre (opción más popular). Podemos ajustar el número de muestras (RF-6).

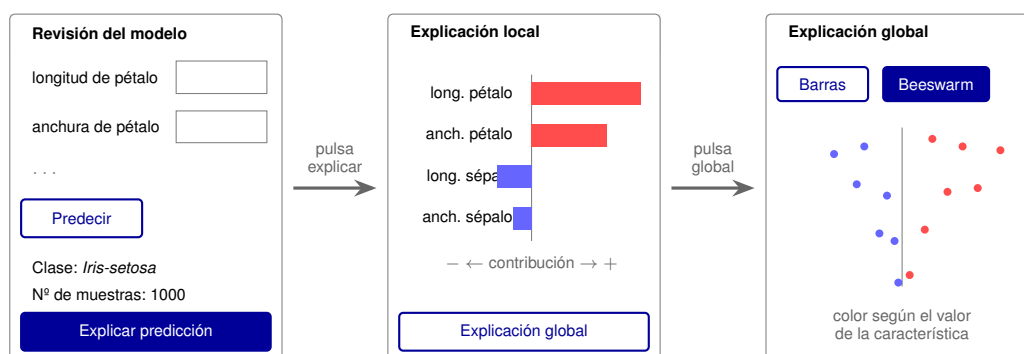


Figura 5.3: *Storyboard* de la explicación tabular: predicción, explicación local y explicación global.

5.2.3. Implementación

La función predictora construye un tensor con las instancias que recibe, evalúa el modelo y devuelve sus predicciones [5.2.3](#).

Función predictora para modelos tabulares

```

1 return async (x) => {
2   const inputTensor = tfjs.tensor2d(x, [x.length, x[0].length]);
3   const predictionTensor = model.predict(inputTensor);
4   const predictionData = await predictionTensor.array();
5   inputTensor.dispose();
6   predictionTensor.dispose();
7   return predictionData;
8 };

```

El resultado es un valor de contribución por cada característica, que se presenta al usuario mediante un gráfico de barras con una barra por característica de la clase predicha, ordenadas de mayor a menor importancia. La altura y el signo de cada barra

indican de un vistazo cuánto y en qué sentido empuja cada atributo a la predicción.

Para la predicción global usabamos un Beeswarm o gráfico de enjambre, (*beeswarm*) [7], donde cada punto es una instancia colocada según su valor SHAP, las filas son las características ordenadas por importancia media y el color de cada punto codifica el valor de la característica, de azul (bajo) a rojo (alto).

5.2.4. Plan de pruebas

Para asegurar el correcto funcionamiento del incremento debemos asegurarnos de que el muestreo funciona de forma correcta, dado que se debe asegurar que los datos que se entregan a SHAP son correctos. Para estas pruebas usamos pruebas unitarias con Vitest. La Tabla 5.1 resume los casos comprobados, con un resultado de nueve pruebas superadas de nueve.

Función	Caso comprobado	Resultado
dataframeRowsToNumb	Convierte celdas de texto y numéricas a número.	OK
	Devuelve una lista vacía ante una entrada que no es un array.	OK
sampleRowsWithoutRe	Devuelve exactamente n filas cuando $n \leq$ tamaño del conjunto.	OK
	No devuelve más filas que las disponibles.	OK
	Las filas devueltas son distintas y pertenecen al conjunto.	OK
buildShapBackground	Muestrea filtrando las filas con el número correcto de características.	OK
	Genera un <i>background</i> de ceros si el conjunto está vacío.	OK
	Genera un <i>background</i> de ceros si ninguna fila tiene el número pedido.	OK
	Nunca muestrea más filas de las solicitadas.	OK

Tabla. 5.1: Pruebas unitarias sobre las utilidades de muestreo de SHAP.

En cuanto a la validación funcional, hemos comprobado sobre la aplicación real que el gráfico de importancia da más peso a los atributos que son más determinantes, y que la explicación global es coherente con las explicaciones locales individuales.

5.3. Incremento 2: explicabilidad en regresión

5.3.1. Objetivo del incremento

Este incremento realmente consiste en extender lo realizado en clasificación tabular a Regresión. Este es el más sencillo dado que a efectos prácticos el valor que tiene un Input para SHAP es realmente las características y sus valores, cosa que es igual tanto para clasificación tabular como para Regresión. Planteamos las mismas historias de uso que el anterior.

- **HU-1:** como estudiante, quiero solicitar una explicación de una predicción, para entender por qué el modelo ha tomado esa decisión.
- **HU-2:** como estudiante, quiero ver qué características influyen más en una predicción sobre datos tabulares, para identificar los atributos más relevantes.

5.3.2. Diseño e implementación

La regresión se trata exactamente igual que la clasificación tabular. La única diferencia es que la salida del modelo es un valor continuo en lugar de una distribución de probabilidad entre clases, así que reutilizamos sin ningún cambio la función predictora tabular del Código 5.2.3 y toda el proceso de KernelSHAP. Lo mismo ocurre con la interfaz, dado que la tarjeta de explicabilidad es la misma que diseñamos en el incremento anterior (Figura 5.3).

5.3.3. Plan de pruebas

Dado que la implementación es idéntica al incremento anterior, las pruebas unitarias son las mismas. Finalmente, lo único a comprobar es la validación funcional, es decir, asegurarnos de que las contribuciones atribuidas por SHAP son coherentes con un modelo de regresión.

5.4. Incremento 3: explicabilidad en clasificación de imágenes

5.4.1. Objetivo del incremento

Este incremento introduce la explicabilidad en imágenes. Cubre las siguientes historias de usuario:

- **HU-3:** como estudiante, quiero ver resaltadas sobre la imagen las regiones que han motivado la predicción, para saber en qué se fija el modelo.
- **HU-5:** como estudiante, quiero ajustar los parámetros de la explicación, para equilibrar la calidad del resultado con el tiempo de cálculo.

5.4.2. Diseño detallado

A diferencia del caso tabular, cuando se trata con imágenes no tiene sentido tratar cada píxel como una característica independiente, es por eso que debemos segmentar la imagen en superpíxeles (regiones) con SLIC con el fin de poder tratar regiones como características, pasando de millones de características que puede tener una imagen a un número reducido alrededor de 7-10. Se presenta un mapa de calor superpuesto a la imagen para poder entender la contribución de cada región en la que se ha segmentado la imagen.

A continuación tenemos un diagrama [5.4](#) que representa como funciona este incremento. Debemos tener en cuenta que tratamos con vectores binarios de regiones de la imagen por lo que va procesando la imagen perturbada al desactivar o activar diferentes regiones de la imagen.

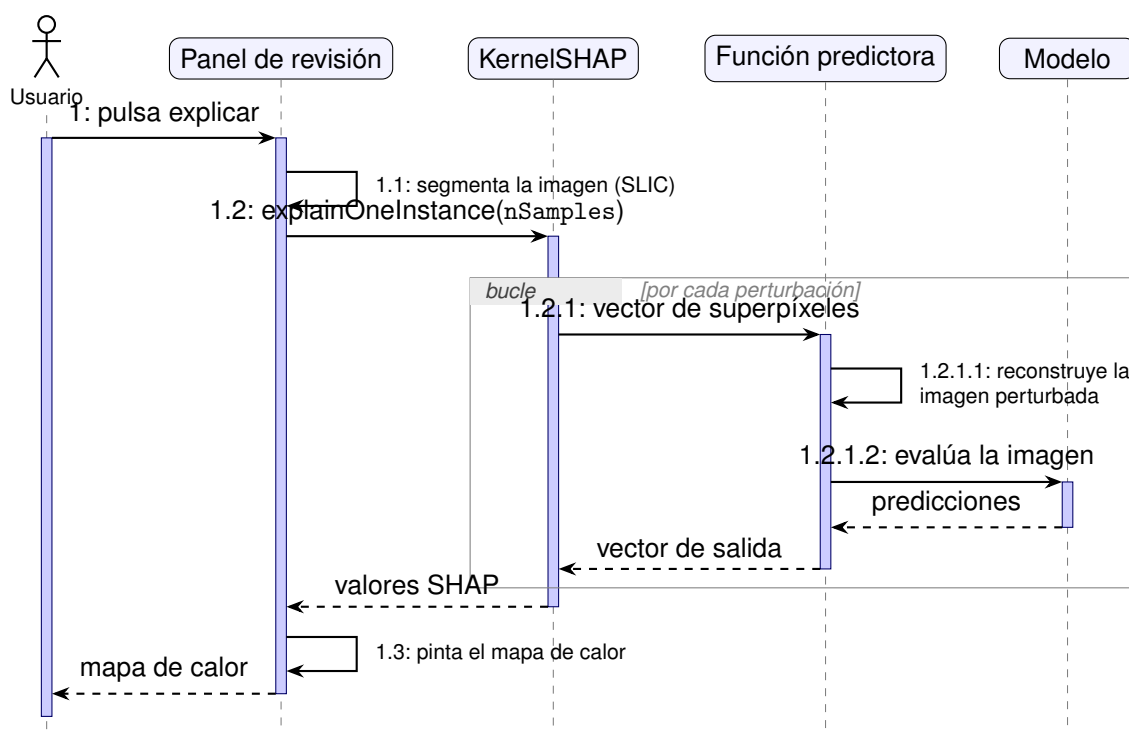


Figura 5.4: Diagrama de secuencia de la explicación de una clasificación de imágenes.

La pantalla donde vive todo esto vuelve a ser la tarjeta de explicabilidad de la revisión del modelo, cuyo *storyboard* se recoge en la Figura 5.5. Además del botón para lanzar la explicación, la tarjeta incorpora los dos parámetros que controlan el equilibrio entre calidad y tiempo de cálculo (HU-5), que son el lado de la rejilla de segmentación y el número de muestras. Como apoyo didáctico, mientras se calcula la explicación vamos guardando algunas de las imágenes perturbadas que evalúa el modelo y las mostramos en una pequeña galería, de forma que el usuario puede ver con sus propios ojos qué le estamos enseñando al modelo, y el resultado final se presenta como un mapa de calor por cada clase considerada.

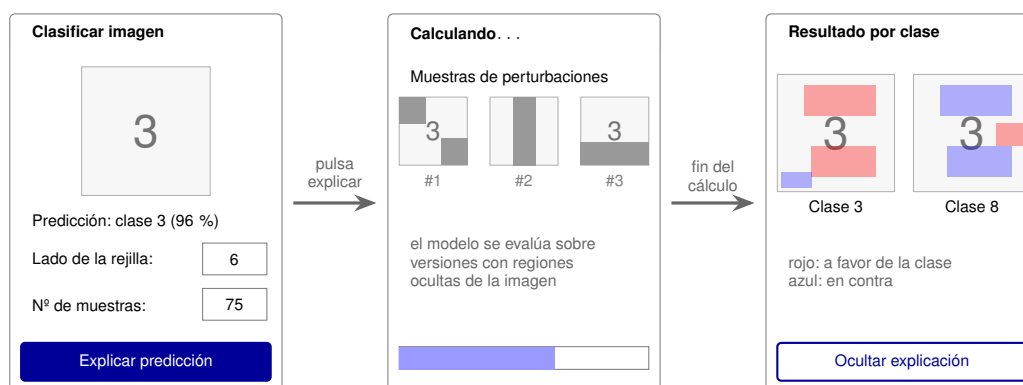


Figura 5.5: *Storyboard* de la explicación de imágenes: parámetros, cálculo con la galería de perturbaciones y mapas de calor por clase.

5.4.3. Implementación

Las entradas que maneja KernelSHAP no son píxeles, sino vectores binarios que indican qué superpíxeles están activos. La función predictora reconstruye, a partir de cada vector, una versión perturbada de la imagen que mantiene visibles las regiones activas y enmascara el resto. El Código 5.4.3 muestra el núcleo de este proceso.

Cálculo de una explicación con KernelSHAP

```
1 return async (x) => {
2   const batchVectors = [];
3   for (let i = 0; i < x.length; i++) {
4     // x[i] es el vector que indica qué superpíxeles están activos
5     const inputTensor = tfjs.tidy(() => {
6       const values = tfjs.tensor1d(Array.from(x[i]).flat());
7       // Se proyecta el vector de superpíxeles sobre el mapa de segmentación
8       const maskFlat = values.gather(segmentationTensor);
9       const mask3d = maskFlat.reshape([height, width, 1]);
10      const mask = mask3d.mul(maskMul).add(maskValue);
11      return imgToTensor.mul(mask).toInt();
12    });
13
14    // El tensor perturbado se convierte de nuevo en imagen para el modelo
15    await tfjs.browser.toPixels(inputTensor, reusableCanvas);
16    const perturbedImageData = reusableCtx.getImageData(0, 0, width, height);
17
18    // Se consulta al modelo a través del contrato común
19    const result = await iModel.CLASSIFY_IMAGE(modelInstance, perturbedImageData);
20    batchVectors.push(toFixedVector(result?.predictions, selectedLabels));
21
22    inputTensor.dispose();
23  }
24  return batchVectors;
25};
```

El uso de `tfjs.tidy` y de `dispose` es de suma importancia, porque cada explicación implica un número muy elevado de evaluaciones y, sin una gestión cuidadosa de los tensores, la memoria de la GPU se agotaría rápidamente en el navegador.

Una vez obtenidas las contribuciones, la visualización se resuelve dibujando sobre un *canvas* la imagen original y superponiéndole una capa de color. Para cada píxel

miramos a qué superpíxel pertenece y recuperamos el valor de contribución de ese segmento. El color representa el signo (rojo para las contribuciones a favor de la clase, azul para las que van en contra) y la transparencia es proporcional a la magnitud, normalizada respecto al valor absoluto máximo, con una opacidad base en torno al 60 %. El Código 5.4.3 muestra el núcleo de ese pintado.

Núcleo del mapa de calor

```
1 // Para cada pixel, buscamos su superpixel y su contribucion
2 const segmentId = map[i];
3 const shapVal = valuesToPaint[segmentId];
4 const absVal = Math.abs(shapVal);
5
6 // Ignoramos el ruido: por debajo del 5 por ciento del máximo no se pinta
7 if (absVal < maxAbsValue * 0.05) continue;
8
9 // Intensidad proporcional a la magnitud; color segun el signo
10 const alpha = (absVal / maxAbsValue) * opacity; // opacity 0.6
11 const r = shapVal > 0 ? 255 : 0; // rojo -> contribucion positiva
12 const b = shapVal > 0 ? 0 : 255; // azul -> contribucion negativa
13
14 // Mezcla alfa sobre el pixel original de la imagen
15 data[idx] = data[idx] * (1 - alpha) + r * alpha;
16 data[idx + 1] = data[idx + 1] * (1 - alpha);
17 data[idx + 2] = data[idx + 2] * (1 - alpha) + b * alpha;
```

5.4.4. Plan de pruebas

La validación funcional ha consistido en comprobar que el mapa de calor resalta las regiones de la imagen realmente asociadas a la clase predicha, y no el fondo ni otras zonas irrelevantes. También hemos verificado el efecto del parámetro `nSamples`: al aumentarlo, la explicación se estabiliza a costa de un mayor tiempo de cálculo.

5.5. Incremento 4: explicabilidad en detección de objetos

5.5.1. Objetivo del incremento

El cuarto incremento adapta la explicabilidad a la detección de objetos, que es la tarea más delicada de todas. Cubre de nuevo la historia de usuario HU-3, aplicada ahora a los modelos de detección:

- **HU-3:** como estudiante, quiero ver resaltadas sobre la imagen las regiones que han motivado la predicción, para saber en qué se fija el modelo.

5.5.2. Diseño detallado

La dificultad de esta tarea es que la salida de un detector no es un vector de probabilidades de tamaño fijo como en la clasificación, sino una lista variable de detecciones, cada una con su caja, su clase y su confianza de forma que de una imagen a otra el detector puede devolver un número distinto de objetos. Como KernelSHAP necesita comparar las predicciones de unas perturbaciones con otras y para ello requiere una salida de longitud constante, fijamos primero, a partir de las detecciones de la imagen original, el conjunto de etiquetas que nos interesan, y para cada perturbación normalizamos la salida del detector a un vector de esa misma longitud (NORMALIZE_PREDICTIONS). Si en una perturbación el modelo deja de detectar algo, esa posición queda a cero.

La Figura 5.6 resume el flujo completo del incremento, incluyendo la decisión de qué segmentación emplear en cada caso, que detallamos en el apartado de implementación.

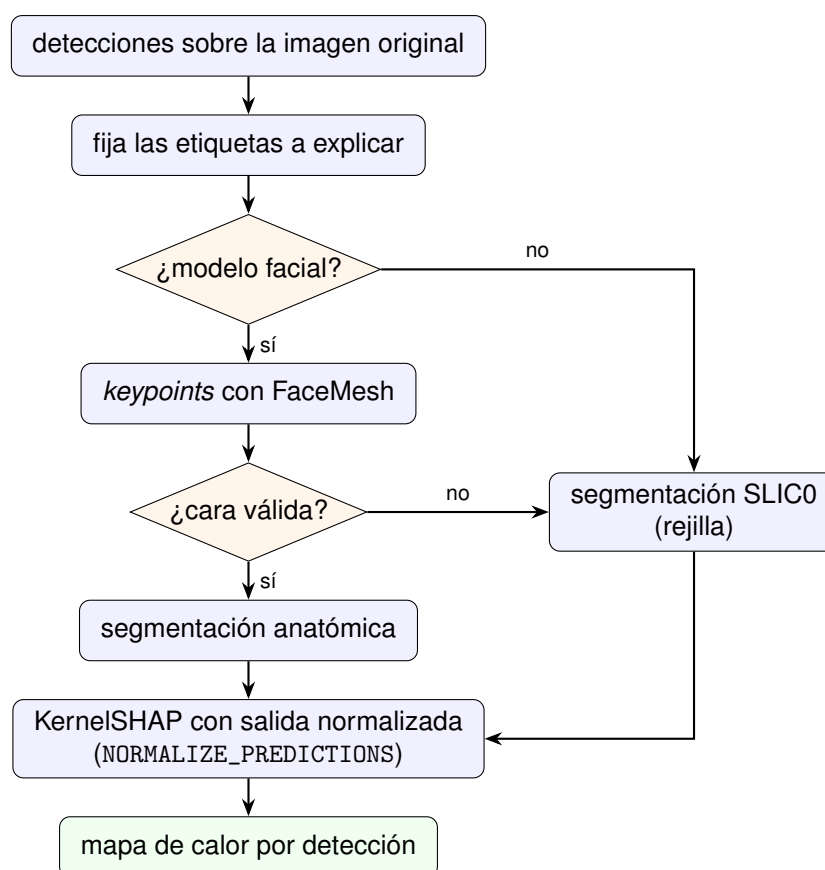


Figura 5.6: Flujo de una explicación en detección de objetos, con la elección de la segmentación.

5.5.3. Implementación

La segmentación también es distinta en este incremento, porque no siempre usamos SLIC. Para los detectores generales partimos de una segmentación en rejilla regular mediante SLIC0, pero cuando el modelo es facial aprovechamos que disponemos de un modelo de malla facial (FaceMesh) para obtener los puntos característicos (*keypoints*) de la cara y construir con ellos una segmentación anatómica de manera que los superpíxeles se corresponden con regiones reales de la cara en lugar de con cuadrados arbitrarios. Como respaldo, si no se detecta ninguna cara o la segmentación sale degenerada, volvemos automáticamente a SLIC0. El Código 5.1 recoge esta elección.

```

1 if (model.faces) {
2   // Modelos faciales: keypoints de FaceMesh -> segmentacion anatomica
3   const faceMesh = await getFaceMeshInstance();
4   const mesh = await faceMesh.PREDICTION(imageData, { flipHorizontal });
5   ({ mapArray, numSegments } = getFaceSegmentMap(

```

```
6     mesh[0].keypoints, imageData.width, imageData.height, flipHorizontal
7     ,
8     ));
9
10    // Respaldo: si no hay cara util, caemos a una rejilla SLIC0
11    if (!numSegments || numSegments <= 1) {
12        ({ mapArray, numSegments } = computeSLICzeroMap(imageData, gridSide)
13        );
14    }
15    } else {
16        // Resto de detectores (COCO-SSD, etc.): rejilla SLIC0
17        ({ mapArray, numSegments } = computeSLICzeroMap(imageData, gridSide));
18    }
```

Listado 5.1: Elección de la segmentación en detección de objetos: keypoints faciales con respaldo en SLIC0

Por último, también cambia la manera de enmascarar las regiones. En la clasificación nos basta con oscurecer los superpíxeles desactivados, pero en la detección —y sobre todo en las caras— oscurecer del todo una región puede confundir al detector, porque una cara llena de parches negros deja de parecer una cara. Por eso, en lugar del oscurecido simple, damos la opción de mezclar la imagen original con una versión desenfocada de sí misma, obtenida aplicando varias pasadas de *average pooling*, de forma que las regiones desactivadas se difuminan en vez de desaparecer y conseguimos perturbaciones mucho más naturales. El resto del proceso es idéntico al de la clasificación de imágenes y el resultado vuelve a mostrarse como un mapa de calor.

Todas estas opciones tienen su reflejo en la interfaz, cuyo *storyboard* se recoge en la Figura 5.7. La tarjeta de explicabilidad sigue el mismo diseño que en la clasificación de imágenes, pero añade los controles propios de la máscara de perturbación, de forma que el usuario puede elegir entre oscurecer o difuminar las regiones desactivadas y, en este último caso, ajustar el tamaño del desenfoque y el número de pasadas.

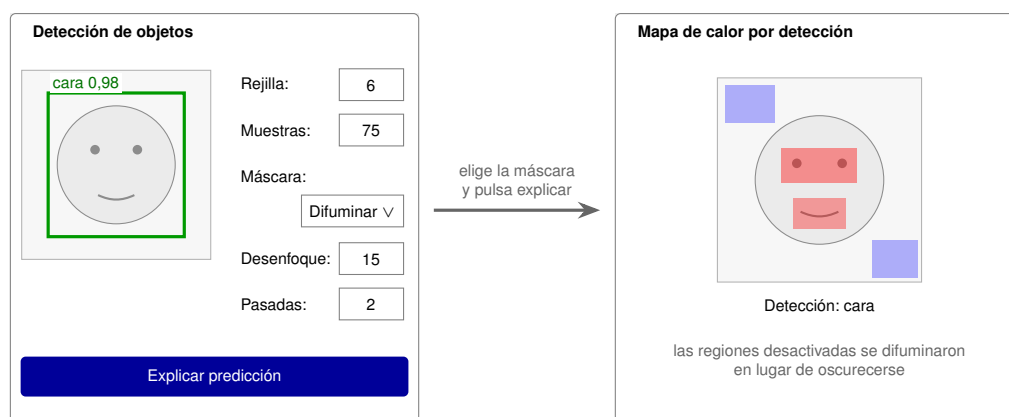


Figura 5.7: *Storyboard* de la explicación en detección de objetos: detección original con los controles de la máscara y mapa de calor resultante.

5.5.4. Plan de pruebas

La validación funcional ha comprobado que, tanto en los detectores generales como en los faciales, el mapa de calor resalta las regiones que realmente motivan cada detección, y que el respaldo a SLIC0 entra correctamente cuando la segmentación facial no es viable.

5.6. Incremento 5: explicabilidad específica mediante LRP

5.6.1. Objetivo del incremento

El último incremento incorpora un método completamente distinto, la explicabilidad específica mediante LRP, para los modelos de imagen cuya arquitectura es accesible. A partir de aquí el usuario puede contrastar la explicación agnóstica de SHAP con la específica de LRP, así que cubre la siguiente historia de usuario:

- **HU-4:** como estudiante, quiero poder elegir entre distintos métodos de explicación, para comparar sus resultados y comprender sus diferencias.

Cuando hablamos de los métodos más importantes dentro de la Inteligencia Artificial Explicable, LRP (*Layer-wise Relevance Propagation*) es sin duda esencial [8]. Se

basa en descomponer la salida del clasificador propagándola hacia atrás por toda la red de manera que, empezando por la predicción, vamos repartiendo una puntuación de relevancia entre las características de entrada (los píxeles) hasta saber cuánto ha contribuido cada una a la decisión final.

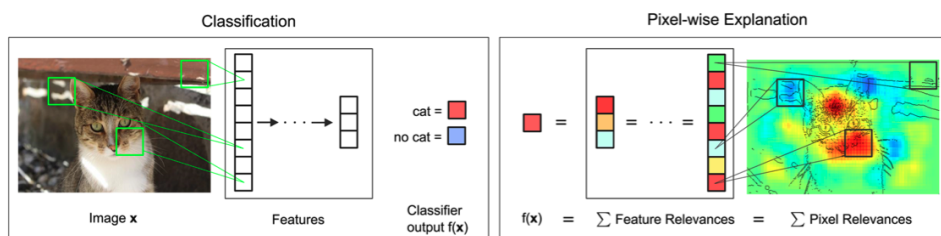


Figura 5.8: Ilustración del funcionamiento de LRP.

5.6.2. Diseño detallado: fundamentos matemáticos

A diferencia de los métodos basados en el gradiente, que solo tienen en cuenta el entorno inmediato del punto de predicción y pueden quedarse atascados en óptimos locales poco representativos, LRP se apoya en un principio estricto de conservación de la relevancia de manera que la evidencia que sostiene una predicción no aparece de la nada ni se pierde por el camino. Este principio nos dice que, en cada capa, la suma de las relevancias que entran tiene que coincidir con la suma de las que salen hacia la capa anterior:

$$\sum_i R_{i \leftarrow j}^{(l, l+1)} = R_j^{(l+1)} \quad (5.1)$$

Así conseguimos que toda la relevancia asociada a la predicción final se vaya repartiendo hacia atrás hasta llegar a los píxeles de entrada, sin que se genere ni se disipe a medida que la señal recorre la red [8].

La otra parte del diseño detallado es la interfaz. Este incremento no crea ninguna pantalla nueva, sino que añade un selector de método en la cabecera de la propia tarjeta de explicabilidad, de forma que el usuario alterna entre SHAP y LRP con un par de botones y puede comparar ambos resultados sobre la misma imagen (HU-4). Cuando el modelo seleccionado no es introspectable, el botón de LRP aparece deshabilitado (RF-7), tal y como recoge el boceto de la Figura 5.9.

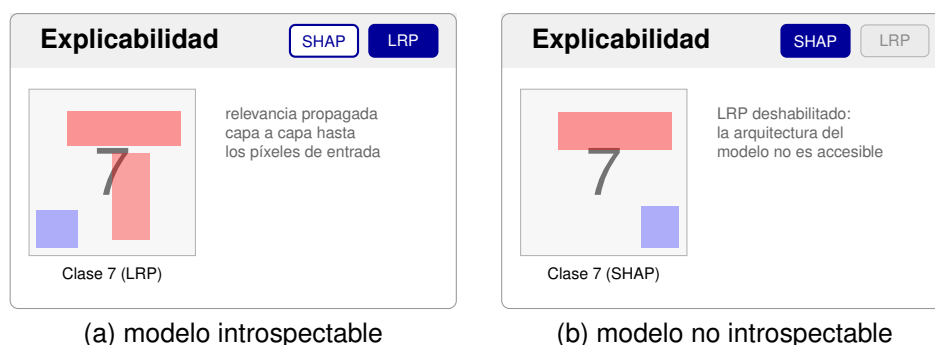


Figura 5.9: Boceto del selector de método en la tarjeta de explicabilidad: con LRP disponible (a) y deshabilitado por no ser el modelo introspectable (b).

5.6.3. Implementación de las reglas de propagación

Al implementar la propagación hacia atrás no usamos una única fórmula, sino varias, porque cada tipo de capa tiene sus particularidades y no todas se comportan igual de bien numéricamente.

Regla épsilon (ϵ)

En las capas densas se realizan muchísimos cálculos y es fácil que aparezca ruido numérico, sobre todo cuando el denominador se acerca a cero. Para evitar esos valores problemáticos, en este tipo de capas hacemos uso de la regla épsilon [3], que introduce un pequeño término estabilizador ϵ en el denominador al calcular la relevancia:

$$R_i = \sum_j \frac{x_i \cdot w_{ij}}{z_j + \epsilon \cdot \text{sign}(z_j)} R_j \quad (5.2)$$

```

1 // Pase hacia adelante matricial: z = x * W + b
2 let z = inputTensor.matMul(weights);
3 if (bias) {
4   z = z.add(bias);
5 }
6
7 // Inyección del estabilizador epsilon respetando el signo del tensor original
8 const stabilizer = tfjs.where(
9   z.greaterEqual(tfjs.scalar(0)),
10  tfjs.scalar(epsilon),
11  tfjs.scalar(-epsilon)
12 );
13 z = z.add(stabilizer);

```



```

14
15 // Retropropagación matemática estabilizada (s = Rout / z)
16 const s = relevanceOut.div(z);
17 const c = s.matMul(weights, false, true);
18
19 // Calculo final de la relevancia de entrada (Rin)
20 const relevanceIn = inputTensor.mul(c);

```

Listado 5.2: Implementación de la regla épsilon (ϵ) en capas densas

Regla alfa-beta ($\alpha\beta$)

En las capas convolucionales, más cercanas a la imagen original, es donde se detectan los bordes, las texturas y las formas, así que aquí no solo nos interesa saber qué apoya la predicción (excitación), sino también qué la contradice (inhibición). Para tratar por separado ambas cosas hacemos uso de la regla $\alpha\beta$ [8], que mantiene diferenciadas las activaciones positivas y negativas y las pondera mediante dos parámetros, α y β :

$$R_{i \leftarrow j}^{(l,l+1)} = R_j^{(l+1)} \cdot \left(\alpha \cdot \frac{z_{ij}^+}{z_j^+} + \beta \cdot \frac{z_{ij}^-}{z_j^-} \right) \quad (5.3)$$

En el código separamos los pesos en su parte positiva (w^+) y su parte negativa (w^-) y propagamos cada una por su lado, para que las contribuciones que suman y las que restan no se mezclen. Para que se siga cumpliendo la conservación de la relevancia trabajamos siempre bajo la restricción $\alpha + \beta = 1$ [8]. El Código 5.3 muestra la regla para las capas densas; en las convolucionales el procedimiento es el mismo, sin más que sustituir el producto matricial por la operación de convolución.

```

1 // Bifurcación del tensor de pesos en excitaciones (wPos) e inhibiciones (wNeg)
2 const wPos = tfjs.maximum(weights, 0);
3 const wNeg = tfjs.minimum(weights, 0);
4
5 // Pase hacia adelante asimétrico y paralelo
6 let zPos = inputTensor.matMul(wPos).add(epsilon);
7 let zNeg = inputTensor.matMul(wNeg).sub(epsilon);
8
9 // Ponderación de la relevancia mediante los hiperparámetros de control
10 const sPos = relevanceOut.div(zPos).mul(alpha);
11 const sNeg = relevanceOut.div(zNeg).mul(beta);
12
13 // Proyección hacia la capa inferior (matrices traspuestas)
14 const cPos = sPos.matMul(wPos, false, true);
15 const cNeg = sNeg.matMul(wNeg, false, true);

```

```
16
17 // Fusión de las evidencias para obtener la relevancia conservada
18 const relevanceIn = inputTensor.mul(cPos.add(cNeg));
```

Listado 5.3: Aplicación de la regla asimétrica $\alpha\beta$

Capas de reducción espacial (*pooling*)

Durante la propagación hacia atrás, LRP también tiene que deshacer los cambios de forma que introducen algunas capas de una red convolucional, como las capas *flatten* o las de *pooling*, porque de lo contrario no podríamos devolver la relevancia a su espacio original. Para el *max pooling*, la formulación estricta de LRP dice que la relevancia debería viajar únicamente hacia la posición de mayor activación, siguiendo un criterio de “el ganador se lo lleva todo” (*winner-takes-all*) [9]. El problema es que recuperar esos índices exactos (*argmax*) resulta muy costoso para la memoria de la GPU (WebGL) dentro del navegador y penalizaría demasiado la experiencia de usuario.

Por ese motivo optamos por una aproximación que se comporta como un agrupamiento promedio, es decir, expandimos el tensor hasta su tamaño original mediante interpolación bilineal (*resizeBilinear*) y repartimos la relevancia a partes iguales entre todos los píxeles que cubre cada región, dividiéndola por el área de la ventana. De este modo seguimos respetando la conservación de la relevancia, pero con un coste asumible en el cliente. El Código 5.4 recoge esta lógica.

```
1 // 1. Deshacer la reducción espacial extrayendo la geometría original
2 const [poolH, poolW] = Array.isArray(poolSize) ? poolSize : [poolSize,
   poolSize];
3
4 // 2. Expansión geométrica mediante interpolación bilineal nativa
5 const resized = tfjs.image.resizeBilinear(relevanceOut, [inH, inW]);
6
7 // 3. Normalización de la relevancia desplegada (conservación)
8 const poolArea = poolH * poolW;
9 const result = resized.div(poolArea);
10
11 return result;
```

Listado 5.4: Despliegue espacial y normalización de la relevancia como aproximación al max pooling

5.6.4. Captura de las activaciones y flujo de ejecución

Como se ve en las reglas anteriores, casi todas necesitan, además de la relevancia que les llega de la capa superior, las activaciones originales de esa capa. Para obtenerlas ejecutamos una inferencia hacia delante capturando el estado interno de la red. El problema es que TensorFlow.js, por su orientación al *front-end*, no nos deja acceder directamente a las activaciones intermedias, así que creamos modelos auxiliares cuya salida son precisamente esos tensores intermedios. Para no crearlos una y otra vez para las mismas capas los guardamos en una caché, tal y como muestra el Código 5.5.

```
1 function _getOrCreateActivationsModel(model, layerNames) {
2   const key = layerNames.join('|');
3   let perModel = _activationsModelCache.get(model);
4
5   if (!perModel) {
6     perModel = new Map();
7     _activationsModelCache.set(model, perModel);
8   }
9
10  let cached = perModel.get(key);
11  if (cached) return cached;
12
13  const outputs = layerNames.map((name) => model.getLayer(name).output);
14  const actModel = tfjs.model({ inputs: model.inputs, outputs });
15
16  cached = { layerNames: [...layerNames], actModel };
17  perModel.set(key, cached);
18
19  return cached;
20 }
```

Listado 5.5: Función de caché para optimizar la extracción de modelos de activación

Con las activaciones guardadas y las reglas implementadas, solo queda ir aplicándolas capa a capa en el orden correcto. Partiendo de la salida de la red, recorreremos sus capas en sentido inverso y, para cada una, miramos de qué tipo es (densa, convolucional, de *pooling* o *flatten*) para decidir qué regla le toca de manera que la relevancia que sale de una capa se convierte en la entrada de la siguiente hasta llegar de nuevo a la imagen original. Este reparto por tipos es justamente el patrón *Strategy* que mencionamos en el Capítulo 4, porque cada capa aplica su propia estrategia de propagación sin que el resto del flujo tenga que enterarse de los detalles. La Figura 5.10 resume este recorrido completo, desde la inferencia inicial que captura las activaciones hasta

la obtención del mapa de relevancia, que se entrega al componente de visualización para pintarlo como un mapa de calor.

5.6.5. Plan de pruebas

Para validar este incremento hemos comprobado que la propagación de relevancia produce mapas coherentes con la predicción sobre los modelos introspectables entrenados en la propia plataforma. Como comprobación adicional, hemos verificado que se cumple de forma aproximada el principio de conservación de la relevancia entre capas, lo que confirma que las reglas de propagación están bien implementadas. Por último, hemos comprobado que el método LRP se deshabilita automáticamente cuando el modelo seleccionado no es introspectable (RF-7).

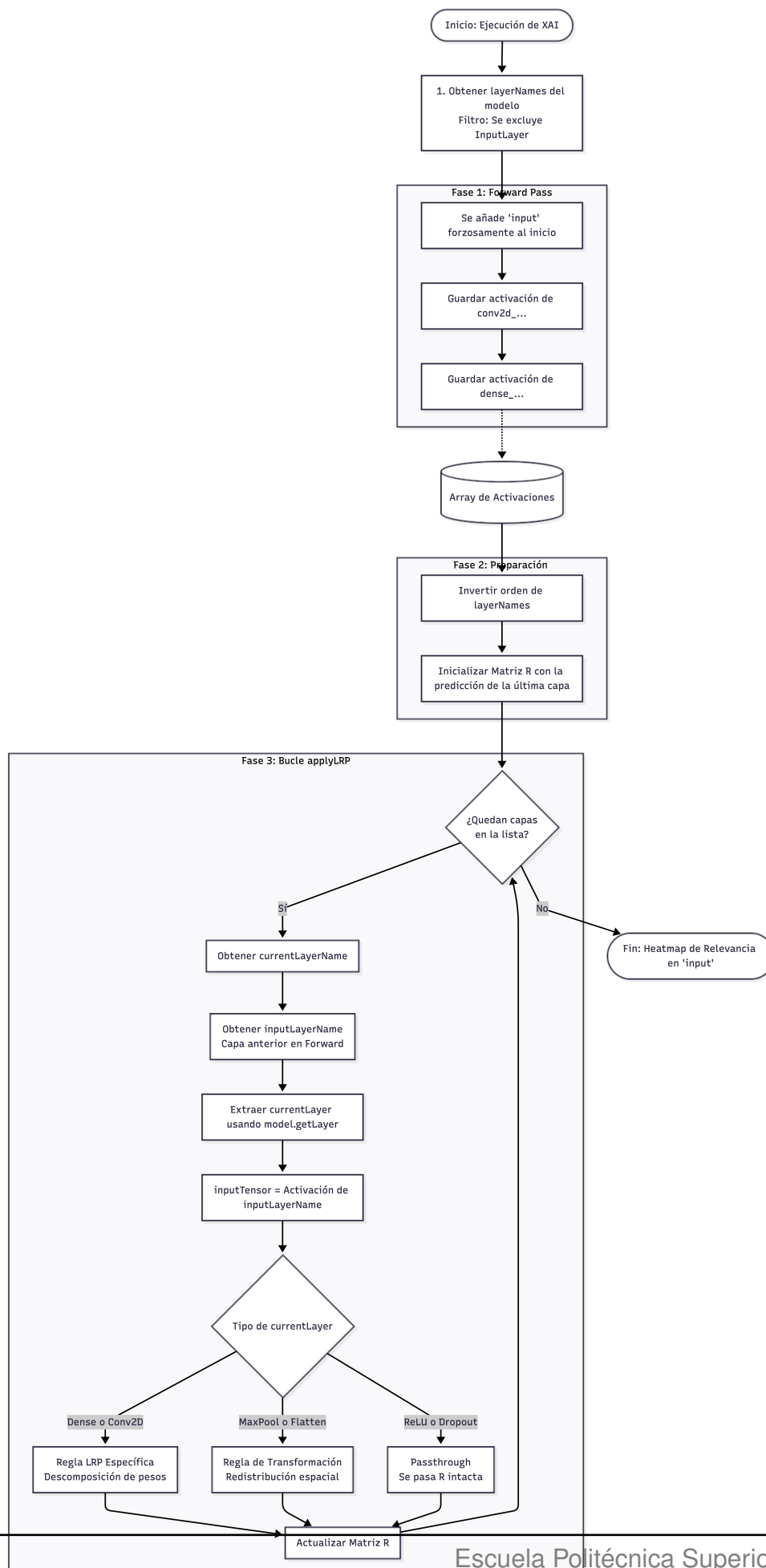


Figura 5.10: Flujo de ejecución de una explicación mediante LRP.

Capítulo 6

Conclusiones

En este último capítulo recogemos las conclusiones del trabajo, contrastando lo que hemos conseguido con los objetivos que nos marcamos al principio y proyectamos una visión a futuro del proyecto.

6.1. Conclusiones

El objetivo general de este Trabajo Fin de Grado era dotar a la plataforma *Nets4Learning* de un módulo de Inteligencia Artificial Explicable que funcionase por completo en el navegador sin depender de ningún servidor, y podemos decir que ese objetivo lo hemos alcanzado ya que la plataforma es ahora capaz de acompañar cada predicción con una explicación visual calculada íntegramente en el lado del cliente.

Si repasamos uno a uno los objetivos específicos que definimos en el Capítulo 1, podemos concluir:

- Se ha **analizado la arquitectura existente** de *Nets4Learning*, comprendiendo cómo la plataforma gestiona sus modelos y realiza la inferencia en cliente con TensorFlow.js lo que ha permitido integrar la nueva capa sin alterar el funcionamiento previo.
- Se han **estudiado y seleccionado las técnicas de XAI viables en el navegador**, contrastando los métodos agnósticos al modelo con los específicos.
- Se ha **implementado la explicabilidad agnóstica (SHAP)** mediante WebSHAP, implementado en todos los modelos a excepción de la detección de números que

utiliza LRP.

- Se ha **implementado la explicabilidad específica (LRP)** sobre el modelo introspectable que utiliza el KMNIST en la detección de números.
- Se han **integrado los componentes en la interfaz** en React de modo que el usuario puede solicitar una explicación y visualizar el resultado como un gráfico de importancia o un mapa de calor.
- Se ha **validado el funcionamiento** del módulo mediante pruebas unitarias y una validación funcional sobre la aplicación real, comprobando que las explicaciones son coherentes con el comportamiento de los modelos (Capítulo 5).

El mayor reto ha sido la restricción de implementación en cliente de forma segura y privada ya que ha condicionado la elección de las técnicas y ha forzado a tomar decisiones comprometidas, como la aproximación que empleamos en las capas de *pooling* de LRP.

En el plano más personal, el proyecto nos ha permitido profundizar en el área de la explicabilidad que apenas se toca durante la carrera, cuya relevancia está todavía por explotar y desarrollarse en diferentes ámbitos como en el de la salud.

6.2. Líneas de trabajo futuro

El módulo si bien es completo, presenta vías de progreso a futuro que mejorarían enormemente el producto.

- **Incorporar nuevas técnicas de explicabilidad**, como Grad-CAM para los modelos convolucionales o LIME. (RNF-5).
- **Ampliar la cobertura de LRP** a una mayor cantidad de arquitecturas y capas complejas.
- **Aprovechar WebGPU** y los nuevos motores de inferencia en cliente con el fin de disipar la limitación de rendimiento de modelos actuales de TensorFlow.
- **Mover el cálculo a Web Workers** de manera que el grueso del trabajo de SHAP se ejecute por completo en un hilo aparte y la interfaz quede aún más liberada durante las explicaciones más largas.

Bibliografía

- [1] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Benne-
tot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Moli-
na, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial
intelligence (xai): Concepts, taxonomies, opportunities and challenges toward res-
ponsible ai. *Information Fusion*, 58:82–115, 2020. doi: 10.1016/j.inffus.2019.12.012.
- [2] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model pre-
dictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volu-
me 30, pages 4765–4774, 2017.
- [3] Grégoire Montavon, Alexander Binder, Sebastian Lapuschkin, Wojciech Samek,
and Klaus-Robert Müller. Layer-wise relevance propagation: an overview. In *Explain-
able AI: Interpreting, Explaining and Visualizing Deep Learning*, pages 193–209.
Springer, 2019.
- [4] Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurelien Lucchi, Pascal Fua,
and Sabine Süsstrunk. SLIC superpixels compared to state-of-the-art superpixel
methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):
2274–2282, 2012. doi: 10.1109/TPAMI.2012.120.
- [5] Jay Wang. Jwebshap: Towards explaining any machine learning models anywhere.
ACM Web Conference 2023, 2023.
- [6] ysjk2003. Superpixel. <https://github.com/>, 2014. Repositorio de GitHub.
- [7] Aidan Cooper. Explaining machine learning models: A non-technical
guide to interpreting shap analyses. [https://www.aidancooper.co.uk/
a-non-technical-guide-to-interpreting-shap-analyses/](https://www.aidancooper.co.uk/a-non-technical-guide-to-interpreting-shap-analyses/), 2021.
- [8] Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen,
Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-
linear classifier decisions by layer-wise relevance propagation. *PloS one*, 10(7):
e0130140, 2015.

- [9] Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, Klaus-Robert Müller, and Wojciech Samek. Layer-wise relevance propagation for neural networks with local renormalization layers. In *Artificial Neural Networks and Machine Learning–ICANN 2016*, pages 63–71. Springer, 2016.

